


# Machine Learning

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br





[1]Faculdade de Sistemas de Informação - UFPA/CUNTINS

21 de outubro de 2026

# Roteiro

Planejamento

Fundamentos de ML

Aprendizado Supervisionado

Projeto de ML

Seleção e Teste

Matriz de Confusão

Regressão

Redes Neurais

Aprendizado Não Supervisionado

Ética e Síntese

Referências



# Licença

Este trabalho está licenciado sob a licença Creative Commons:





# Objetivos de aprendizagem

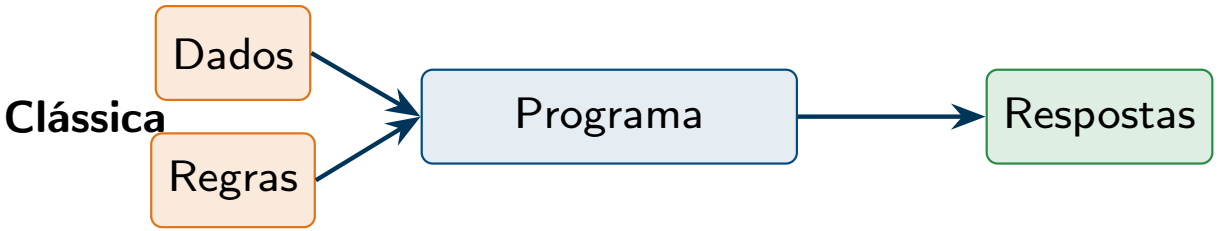
Ao final da aula, você será capaz de:

1. explicar o que é aprendizado de máquina e distinguir seus três paradigmas.
2. treinar e testar um modelo supervisionado sem vazamento de dados.
3. avaliar classificadores e regressores com as métricas adequadas.
4. descrever como uma rede neural aprende, do perceptron à retropropagação.
5. aplicar K-Means e PCA para explorar dados sem rótulos.

Códigos do livro: `https://github.com/aimacode`. Códigos do professor: GitHub.



# Programar regras ou aprender com dados?







Na programação clássica, a pessoa escreve as regras. No aprendizado de máquina, o algoritmo as descobre a partir de exemplos.

# O que é aprender?

## Definição de Mitchell (1997)

Um programa aprende com a experiência $E$, em relação a uma tarefa $T$ e a uma medida de desempenho $P$, se o seu desempenho em $T$, medido por $P$, melhora com $E$.

## Exemplo: filtro de spam

- $T$: classificar e-mails como spam ou não spam.
- $E$: e-mails já rotulados por usuários.
- $P$: porcentagem de e-mails classificados corretamente.

A expressão *machine learning* foi popularizada por Arthur Samuel em 1959, com um programa que aprendia a jogar damas.



# IA, ML e Deep Learning

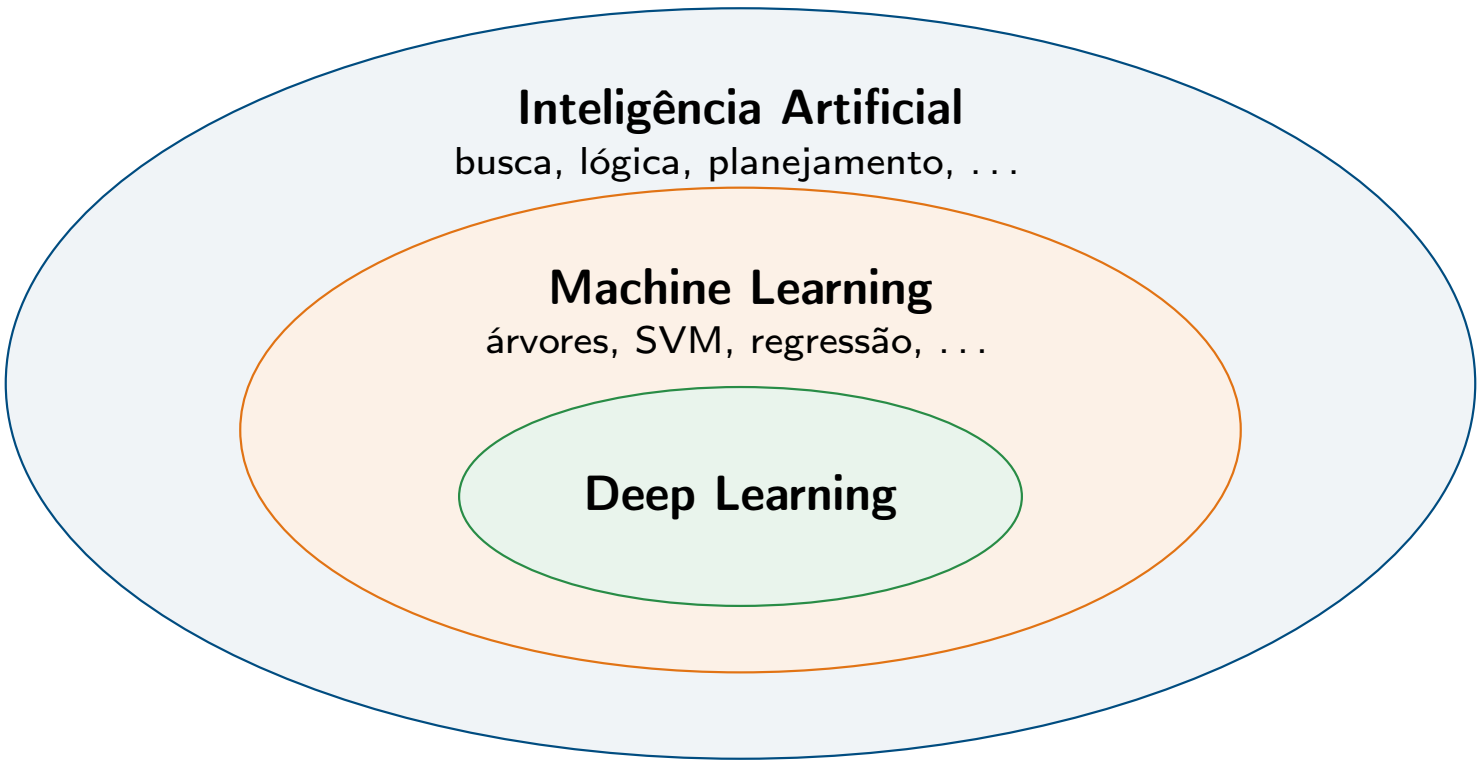
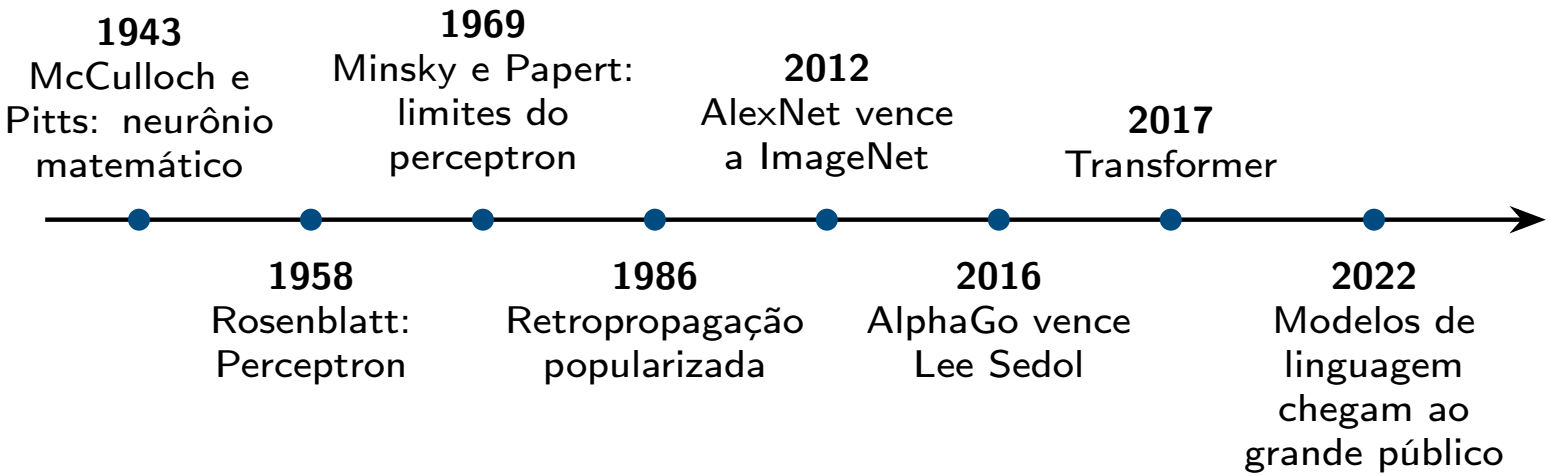



**Figura:** Relação entre IA, *machine learning* e *deep learning*.

# Uma linha do tempo

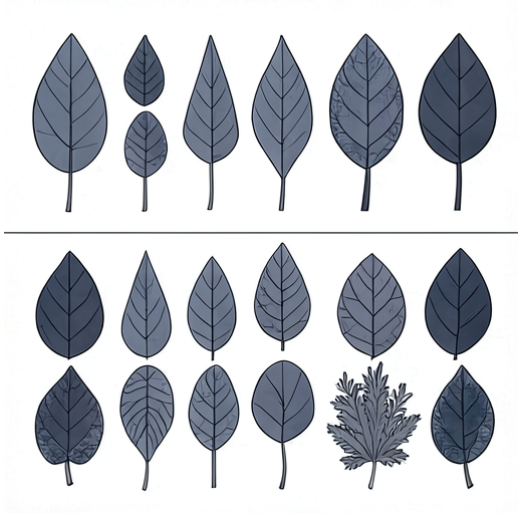


# Onde o ML já está em uso

| Área | Tarefa de ML | Exemplo atual |
|---|---|---|
| Saúde | classificar imagens médicas | triagem de retinopatia diabética |
| Biologia | prever estruturas | AlphaFold prevê a forma de proteínas |
| Linguagem | modelar texto | assistentes, tradução e geração de código |
| Mobilidade | perceber o ambiente | carros autônomos detectam pedestres |
| Finanças | detectar anomalias | fraude em cartões de crédito |
| Mídia | recomendar itens | Netflix e Amazon |



# Três formas de aprender



**Supervisionado**

exemplos com rótulo



**Não supervisionado**

estrutura sem rótulo



**Por reforço**

tentativa, erro e recompensa

**Figura:** Exemplos visuais dos três paradigmas: dados rotulados, amostras agrupadas por semelhança e um braço robótico que aprende por interação.

# Comparando os paradigmas

| | **Supervisionado** | **Não supervisionado** | **Reforço** |
|---|---|---|---|
| Dados | pares $(x, y)$ | apenas $x$ | interação com o ambiente |
| Objetivo | prever $y$ | achar estrutura | maximizar recompensa |
| Exemplo | diagnóstico por imagem | segmentar clientes | robô que aprende a andar |

**Dilema do reforço:** explorar ações novas ou aproveitar a melhor ação conhecida (*exploration vs. exploitation*). É como escolher entre um restaurante novo e o seu favorito.



# Os modelos de linguagem usam qual paradigma?

- Pré-treino **autossupervisionado**: o rótulo é a próxima palavra do próprio texto.
- Ajuste fino **supervisionado** com exemplos de boas respostas.
- Alinhamento com **reforço** a partir de preferências humanas (RLHF).

**? Para refletir**

Por que a autossupervisão permite treinar com bilhões de textos sem rotulagem manual?



# Aprendizado supervisionado

Dado um conjunto de exemplos

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{m},$$

queremos aprender uma função $h$ tal que $h(x) \approx y$ **também para dados novos**.

- $x$: atributos (*features*).
- $y$: rótulo ou valor alvo.
- Classificação: $y$ é uma categoria.
- Regressão: $y \in \mathbb{R}$.

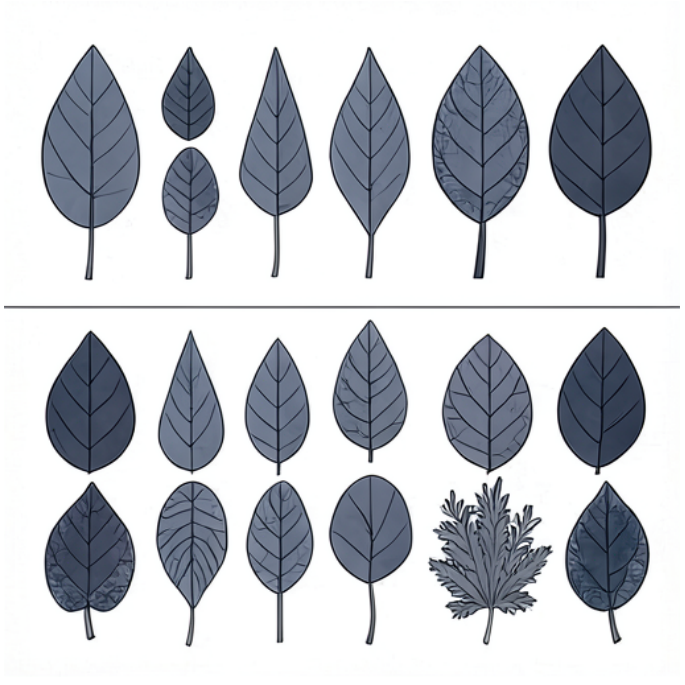


**Figura:** Exemplos já rotulados por espécie.

**Para refletir**

E se todas as folhas de uma espécie no treino vierem do mesmo jardim?

# As peças do aprendizado

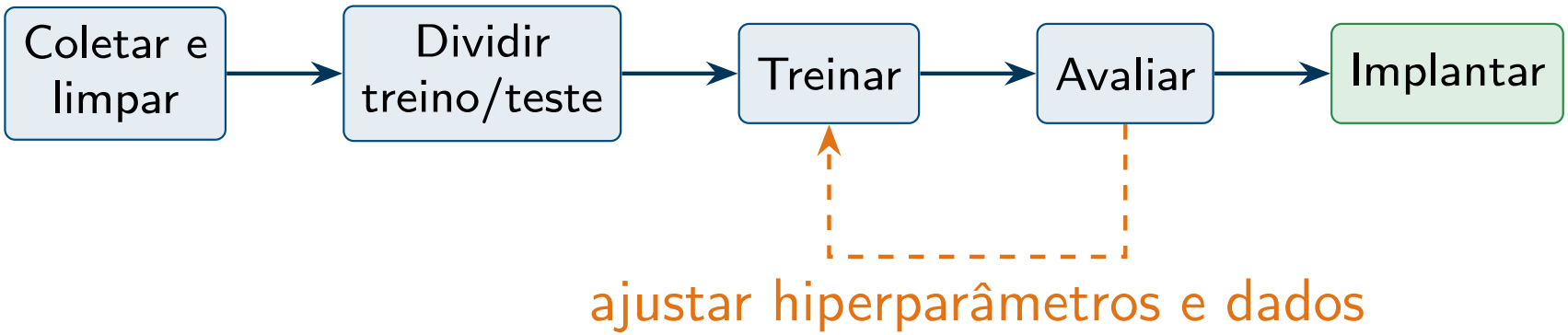
- **Modelo** $h_\theta$: família de funções com parâmetros $\theta$.
- **Perda** $\ell$: mede o erro de uma previsão.
- **Otimização**: escolhe $\theta$ que minimiza o risco empírico

$$\hat{R}(\theta) = \frac{1}{m} \sum_{i=1}^{m} \ell\big(h_\theta(x_i), y_i\big).$$

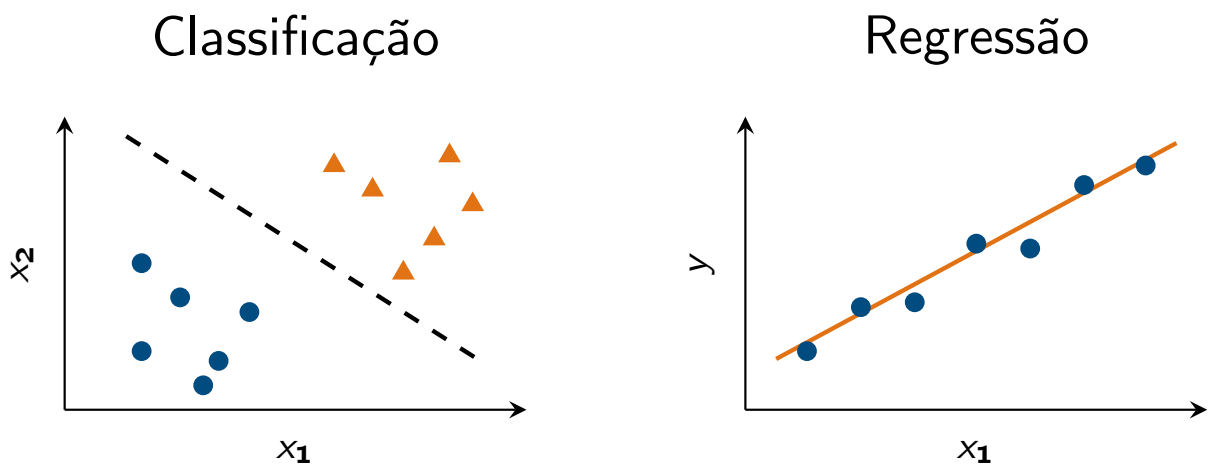
- **Avaliação**: mede o desempenho em dados que o modelo não viu.



# Fluxo de trabalho

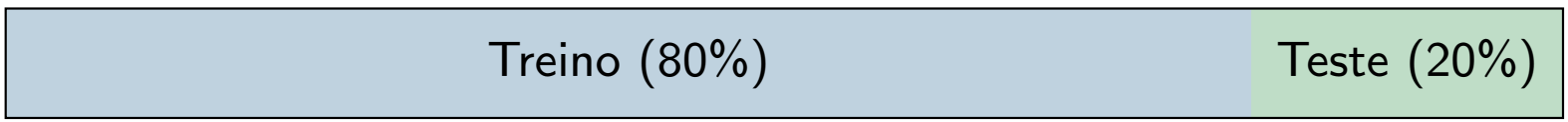


# Classificação e regressão




A saída é uma **categoria** na classificação e um **número** na regressão.

# Treino e teste




- **Treino**: ajusta os parâmetros.
- **Teste**: estima o desempenho final, e deve ser usado uma única vez.

# Underfitting e overfitting

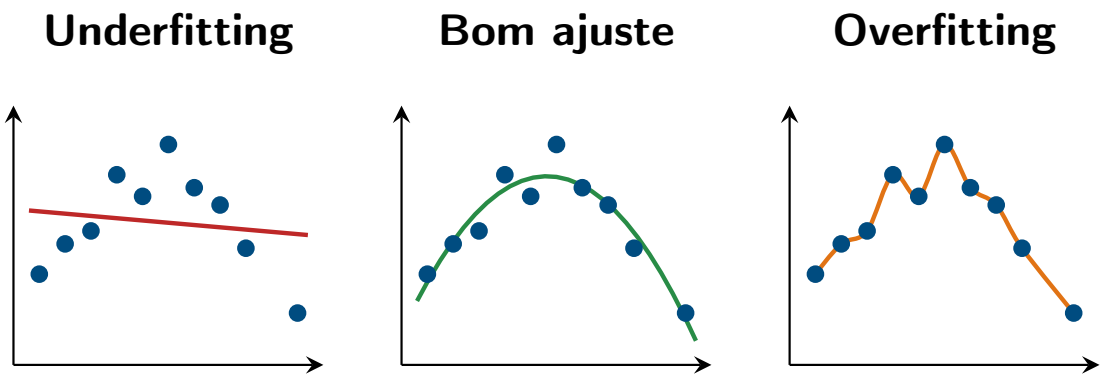


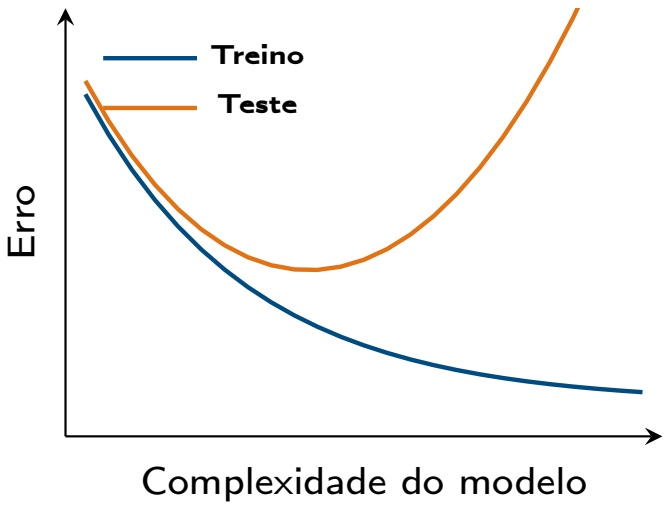

**Figura:** Mesmos dados, três modelos: reta, parábola e curva que passa por todos os pontos.

# Como diagnosticar

| **Situação** | **Erro treino** | **Erro teste** | **O que fazer** |
|---|---|---|---|
| Underfitting | alto | alto | modelo mais flexível, mais atributos |
| Bom ajuste | baixo | baixo | manter e testar |
| Overfitting | baixo | alto | mais dados, regularização, modelo mais simples |



# Viés e variância




Erro esperado com perda quadrática:

$$\text{viés}^2 + \text{variância} + \text{ruído}.$$

- Pouca complexidade: viés alto.
- Muita complexidade: variância alta.
- Buscamos o ponto intermediário.

# Memorizar não é entender

**Como evitar o overfitting**

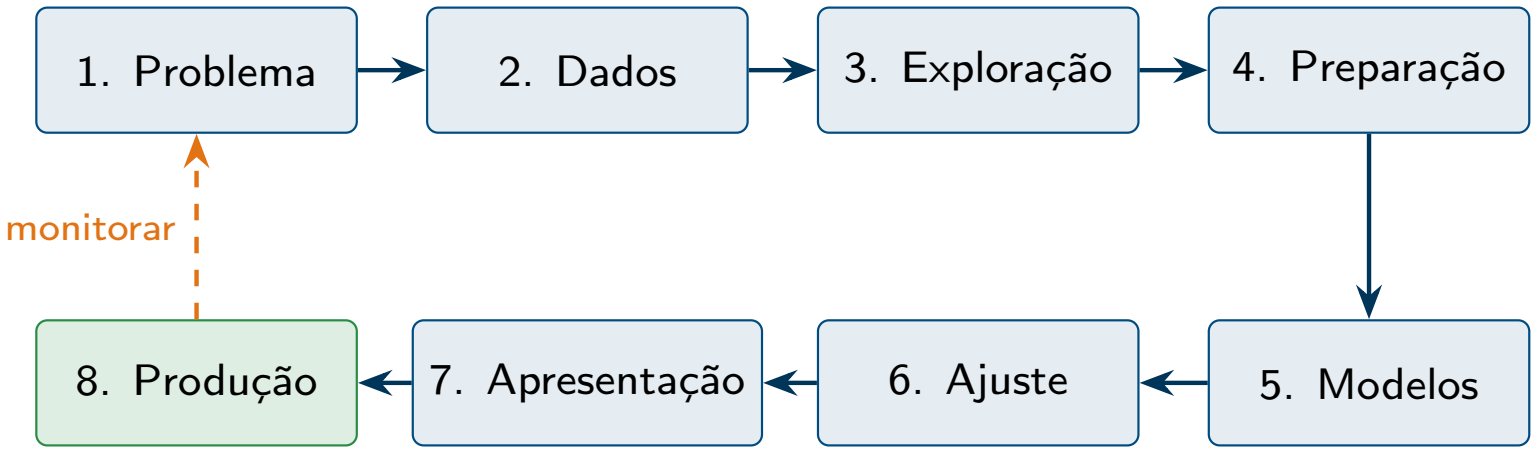
- Mais dados de boa qualidade.
- Modelo mais simples.
- Regularização.
- Cross-validation.

**Para refletir**

Um aluno que decorou a prova de 2023 está preparado para a de 2024?



# As oito etapas de um projeto




O processo é iterativo: a monitoração em produção gera novos dados que reiniciam o ciclo.

# 1. Problema e 2. Dados

**Reflita sobre o problema**

- Qual é o objetivo de negócio?
- Como a solução será usada?
- Como medir o sucesso?
- Existe uma linha de base simples?

**Obtenha os dados**

- Quais dados e em que volume?
- Há restrições legais e de privacidade?
- Separe o conjunto de teste agora e não o toque.



# 3. Exploração e 4. Preparação

**Explore**

- Trabalhe em uma cópia dos dados.
- Estude tipo, ausências e distribuição de cada atributo.
- Visualize e meça correlações.

**Prepare**

- Trate ausências e *outliers*.
- Codifique categorias.
- Padronize atributos numéricos.
- Escreva funções reutilizáveis.



# 5. Modelos, 6. Ajuste e 7. Apresentação

- Treine modelos de famílias diferentes e compare por cross-validation.
- Ajuste os hiperparâmetros dos três a cinco melhores candidatos.
- Combine modelos (*ensembles*) se houver ganho real.
- Documente suposições e limitações antes de apresentar.

**Atenção**

Não ajuste o modelo depois de olhar o erro no teste. Ele deixa de ser uma estimativa honesta.



# 8. Produção

- Conecte o modelo às fontes de dados e escreva testes.
- Monitore o desempenho com dados reais.
- Verifique a qualidade das entradas (sensores com defeito, dados antigos).
- Retreine com dados novos para acompanhar mudanças no mundo (*data drift*).



# Vazamento de dados

**Errado**: normalizar tudo antes de dividir.

```
X = StandardScaler().
    fit_transform(X)
X_tr, X_te = train_test_split(X)
```

O teste influenciou a média e o desvio usados no treino.

**Certo**: aprender a transformação só no treino.

```
X_tr, X_te = train_test_split(X)
pipe = make_pipeline(
    StandardScaler(),
    LogisticRegression())
pipe.fit(X_tr, y_tr)
```



# Algoritmos clássicos

| **Algoritmo** | **Ideia** | **Ponto de atenção** |
|---|---|---|
| k-NN | classe dos $k$ vizinhos mais próximos | lento com muitos dados e sensível à escala |
| Árvore de decisão | perguntas sucessivas sobre atributos | tende ao overfitting |
| *Random Forest* | muitas árvores votando | menos interpretável |
| Naive Bayes | probabilidades com atributos independentes | independência raramente vale |
| SVM | margem máxima entre classes | custo alto em bases grandes |
| Regressão logística | probabilidade por função sigmoide | fronteira linear |



# Exemplo completo com scikit-learn

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

pipe = make_pipeline(StandardScaler(),
                     LogisticRegression(max_iter=1000))
grid = GridSearchCV(
    pipe, {"logisticregression__C": [0.01, 0.1, 1, 10]}, cv=5)
grid.fit(X_tr, y_tr)
print(classification_report(y_te, grid.predict(X_te)))
```



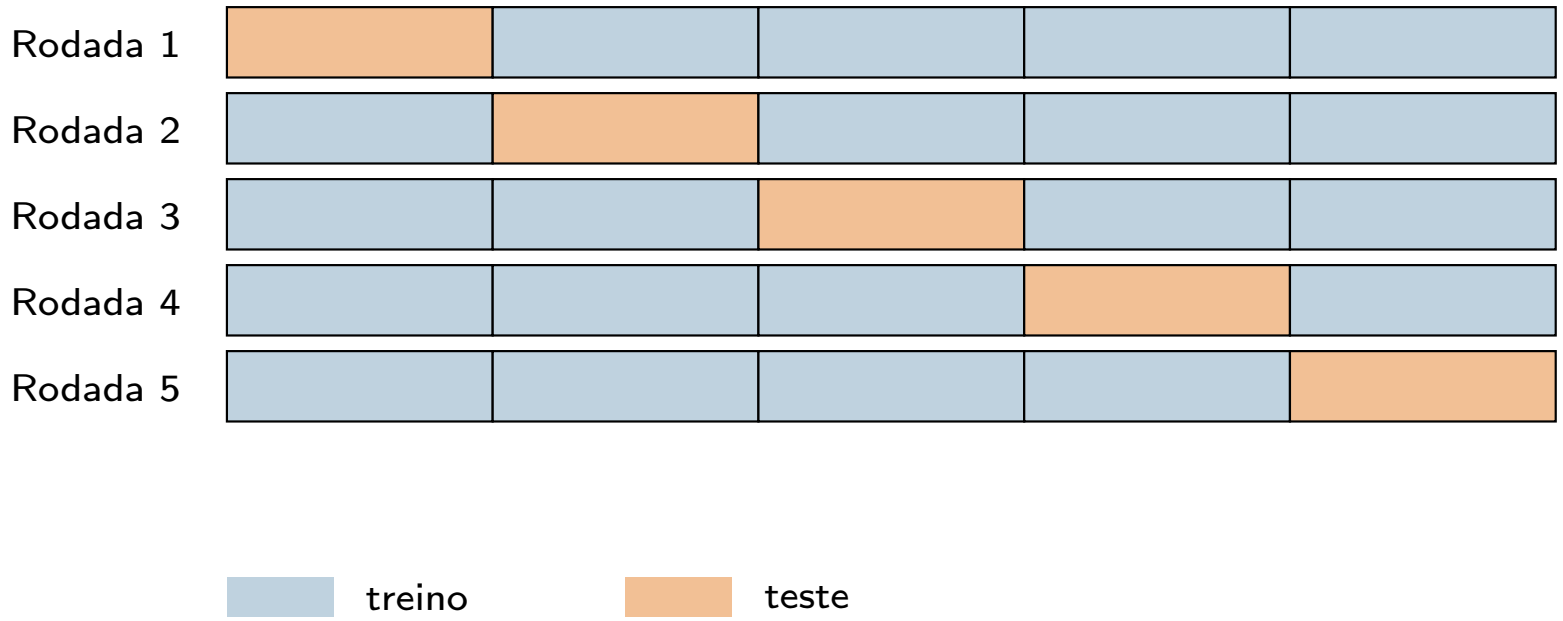
# Seleção de modelos

- **Seleção**: escolher a família de hipóteses (árvores, redes, regressão).
- **Otimização**: achar a melhor hipótese dentro da família escolhida.
- O desafio é equilibrar underfitting e overfitting.

| | **Parâmetros** | **Hiperparâmetros** |
|---|---|---|
| Quem define | o algoritmo, ao treinar | a pessoa, antes de treinar |
| Exemplos | pesos, coeficientes | $k$ do k-NN, profundidade, $\lambda$ |



# Cross-validation com $k$ partições




- Cada partição serve uma vez como teste.
- O resultado é a média das $k$ notas, com seu desvio padrão.

# Cross-validation: um exemplo numérico

Cinco rodadas, acurácias de teste: 0,84, 0,88, 0,80, 0,86, 0,82.

$$\bar{a} = 0{,}84, \qquad s = \sqrt{\frac{0 + 0{,}0016 + 0{,}0016 + 0{,}0004 + 0{,}0004}{4}} \approx 0{,}032.$$

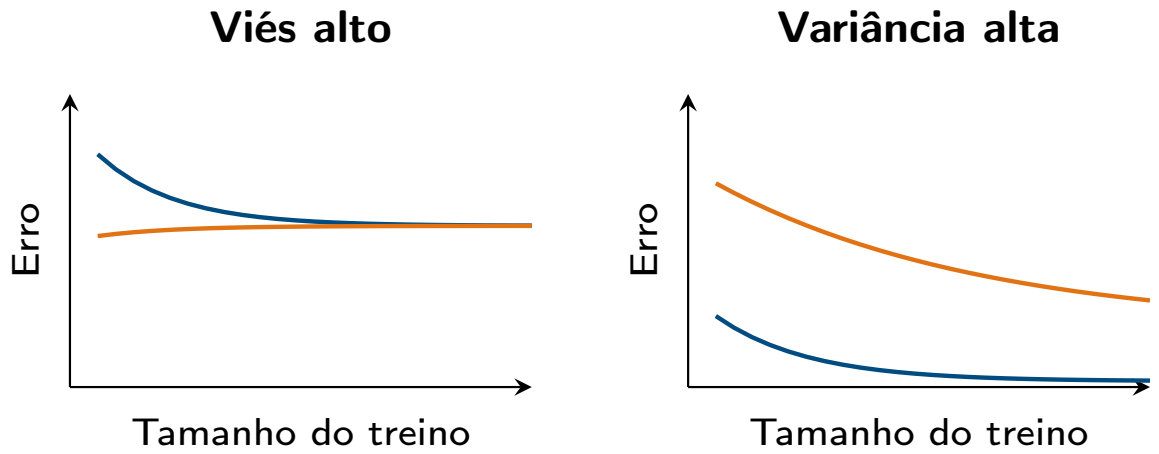
Relatamos $0{,}84 \pm 0{,}03$. A cross-validation reduz a variância da estimativa de desempenho, porque não depende de uma única divisão.

**Para refletir**

Dados de séries temporais podem ser embaralhados antes da divisão? Por quê?



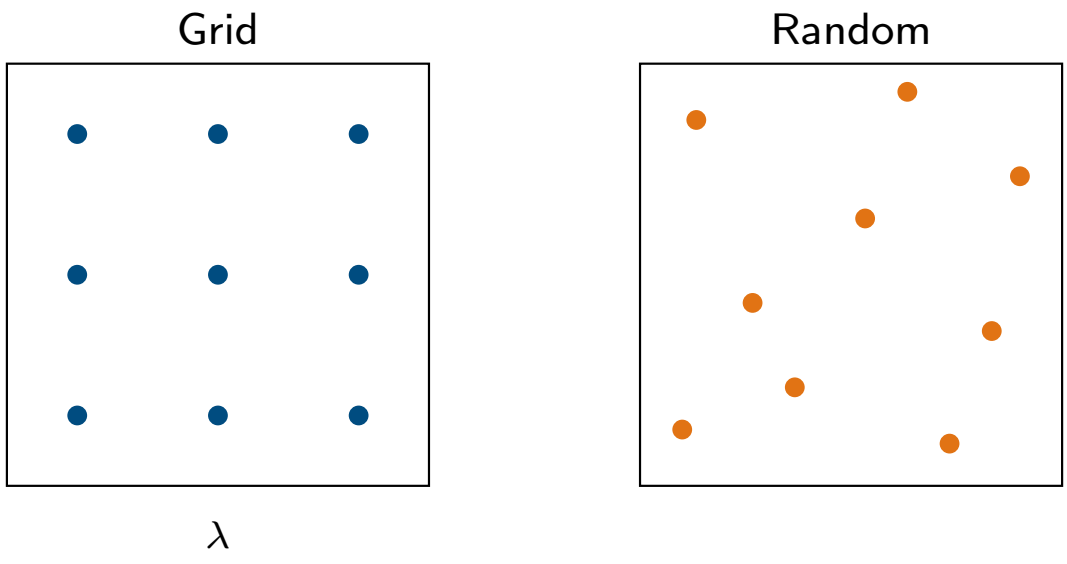
# Curvas de aprendizado




azul: erro de treino. laranja: erro de teste (curvas ilustrativas).

No viés alto, mais dados não ajudam. Na variância alta, mais dados ajudam.

# Busca de hiperparâmetros




Com a mesma quantidade de testes, a random search explora mais valores distintos de cada hiperparâmetro. A otimização bayesiana usa os resultados anteriores para escolher o próximo teste.

# Matriz de confusão

| | | Previsto: Positivo | Previsto: Negativo |
|---|---|---|---|
| Real | Positivo | Verdadeiro Positivo (VP) | Falso Negativo (FN) |
| Real | Negativo | Falso Positivo (FP) | Verdadeiro Negativo (VN) |

A diagonal mostra os acertos. Fora dela estão os dois tipos de erro.



# VP, FN, FP, VN e suas equações

- **VP (verdadeiro positivo)**: positivo previsto e real positivo.
- **FN (falso negativo)**: negativo previsto, mas real positivo.
- **FP (falso positivo)**: positivo previsto, mas real negativo.
- **VN (verdadeiro negativo)**: negativo previsto e real negativo.

$$\text{Acurácia} = \frac{VP + VN}{VP + FN + FP + VN}$$

$$\text{Precisão} = \frac{VP}{VP + FP}$$

$$\text{Recall} = \frac{VP}{VP + FN}$$

$$F_1 = \frac{2 \cdot \text{Precisão} \cdot \text{Recall}}{\text{Precisão} + \text{Recall}}$$

# Exemplo: filtro de spam

| | **Previsto spam** | **Previsto não spam** |
|---|---|---|
| **Real spam** | 50 (VP) | 10 (FN) |
| **Real não spam** | 5 (FP) | 100 (VN) |

| **Métrica** | **Pergunta que responde** | **Valor** |
|---|---|---|
| Acurácia | quanto acertamos no total? | $\frac{150}{165} \approx 90{,}9\%$ |
| Precisão | dos que previmos spam, quantos eram? | $\frac{50}{55} \approx 90{,}9\%$ |
| *Recall* | dos spams reais, quantos achamos? | $\frac{50}{60} \approx 83{,}3\%$ |
| F1 | equilíbrio entre precisão e *recall* | $\frac{2 \cdot 50}{2 \cdot 50 + 5 + 10} \approx 0{,}87$ |



# O paradoxo da acurácia

Em 1000 pacientes, apenas 10 têm a doença. Um modelo que responde sempre "saudável" obtém:

$$\text{acurácia} = \frac{990}{1000} = 99\%, \qquad \textit{recall} = \frac{0}{10} = 0\%.$$

**Para refletir**

Esse modelo é útil? Qual métrica revela o problema?

Com classes desbalanceadas, use *recall*, precisão, F1 e a matriz de confusão.



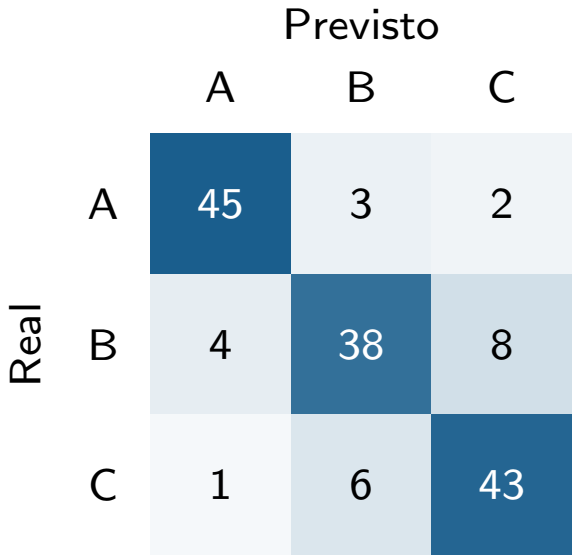
# Qual erro custa mais?

| **Cenário** | **Erro mais grave** | **Priorizar** |
|---|---|---|
| Triagem de câncer | falso negativo (doença não detectada) | *recall* |
| Filtro de spam | falso positivo (e-mail importante perdido) | precisão |
| Detecção de fraude | depende do custo da fraude e da revisão | ambos, via F1 ou custo |

O limiar de decisão controla essa troca. Reduzir o limiar aumenta o *recall* e diminui a precisão.



# Classificação multiclasse




| | Prec. | *Recall* | F1 |
|---|---|---|---|
| A | 0,90 | 0,90 | 0,90 |
| B | 0,81 | 0,76 | 0,78 |
| C | 0,81 | 0,86 | 0,84 |

Acurácia: $126/150 = 84\%$. Média macro do F1: 0,84.

Cada linha soma os exemplos reais da classe. A classe B é a mais confundida.

# Exercício

Um detector de fraude produziu $VP = 30$, $FN = 20$, $FP = 10$ e $VN = 940$.
Calcule acurácia, precisão, *recall* e F1.

## Resposta

Acurácia $= 970/1000 = 97\%$. Precisão $= 30/40 = 0{,}75$. *Recall* $= 30/50 = 0{,}60$. F1 $= 60/90 \approx 0{,}67$.

A acurácia alta esconde que 40% das fraudes passam despercebidas.

# Exercício

Um detector de fraude produziu $VP = 30$, $FN = 20$, $FP = 10$ e $VN = 940$.
Calcule acurácia, precisão, *recall* e F1.

## Resposta

Acurácia $= 970/1000 = 97\%$. Precisão $= 30/40 = 0{,}75$. *Recall* $= 30/50 = 0{,}60$. F1 $= 60/90 \approx 0{,}67$.

A acurácia alta esconde que 40% das fraudes passam despercebidas.

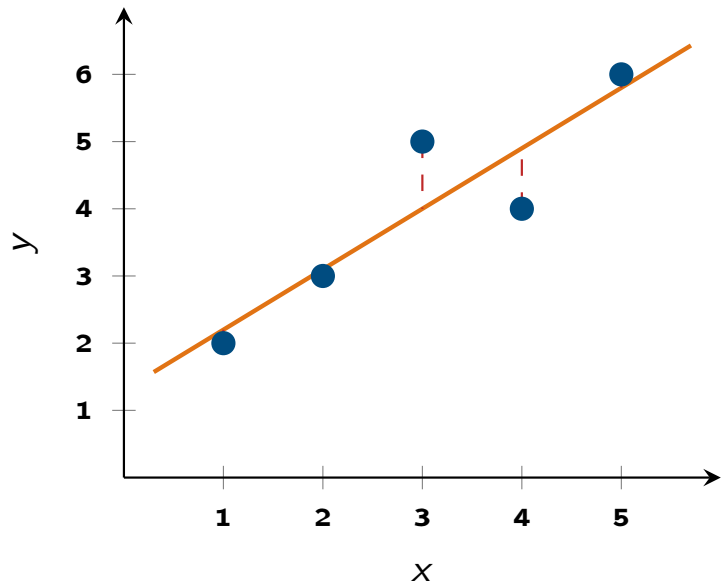


# Regressão linear simples

$$y = \beta_0 + \beta_1 x + \varepsilon$$

- $\beta_0$: intercepto.
- $\beta_1$: variação esperada em $y$ para cada unidade de $x$.
- $\varepsilon$: erro aleatório.

Os coeficientes minimizam a soma dos quadrados dos resíduos (mínimos quadrados).



# Exemplo resolvido

Dados $(x, y)$: $(1,2), (2,3), (3,5), (4,4), (5,6)$. Médias: $\bar{x} = 3$ e $\bar{y} = 4$.

$$\hat{\beta}_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} = \frac{9}{10} = 0{,}9, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} = 1{,}3.$$

| $x$ | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| $y$ | 2 | 3 | 5 | 4 | 6 |
| $\hat{y}$ | 2,2 | 3,1 | 4,0 | 4,9 | 5,8 |
| resíduo | −0,2 | −0,1 | 1,0 | −0,9 | 0,2 |



# Métricas de regressão

| Métrica | Fórmula | No exemplo |
|---|---|---|
| MSE | $\frac{1}{n}\sum(y_i - \hat{y}_i)^2$ | $1{,}90/5 = 0{,}38$ |
| RMSE | $\sqrt{\text{MSE}}$ | 0,62, na unidade de $y$ |
| MAE | $\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$ | $2{,}4/5 = 0{,}48$ |
| $R^2$ | $1 - \frac{\sum(y_i-\hat{y}_i)^2}{\sum(y_i-\bar{y})^2}$ | $1 - \frac{1{,}9}{10} = 0{,}81$ |

- O MSE pune erros grandes mais do que o MAE.
- $R^2$ pode ser **negativo** em dados de teste, quando o modelo é pior que a média.
- $R^2$ ajustado penaliza atributos que não ajudam.



# Regressão múltipla

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \varepsilon \quad \Longleftrightarrow \quad \mathrm{y} = \mathrm{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}.$$

Com $\mathrm{X} \in \mathbb{R}^{n \times (p+1)}$ (primeira coluna de uns), a solução de mínimos quadrados é

$$\hat{\boldsymbol{\beta}} = (\mathrm{X}^\top \mathrm{X})^{-1} \mathrm{X}^\top \mathrm{y}.$$

Cada $\beta_j$ mede o efeito de $x_j$ **mantendo os demais fixos**. Exemplo: o preço de um imóvel cresce $\beta_1$ por metro quadrado, com o número de quartos constante.

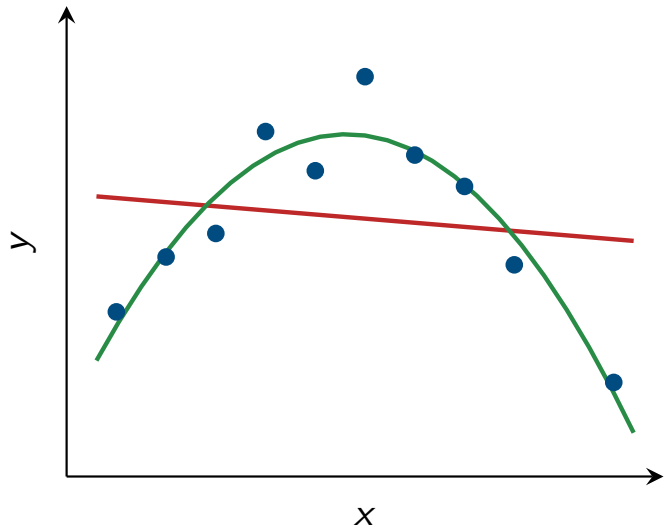


# Regressão polinomial

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_d x^d$$

O modelo é **linear nos coeficientes**, mesmo com curva em $x$. Por isso os mesmos mínimos quadrados se aplicam.

Na figura de underfitting e overfitting, a reta tem SSE $= 11{,}9$ e a parábola $1{,}3$.



# Regularização

Penaliza coeficientes grandes para reduzir a variância:

$$\text{Ridge (L2):} \quad \|y - X\beta\|^2 + \lambda\|\beta\|_2^2$$
$$\text{Lasso (L1):} \quad \|y - X\beta\|^2 + \lambda\|\beta\|_1$$

- Ridge encolhe todos os coeficientes.
- Lasso pode zerar coeficientes e, assim, selecionar atributos.
- Padronize os atributos antes e não penalize o intercepto.
- Escolha $\lambda$ por cross-validation.



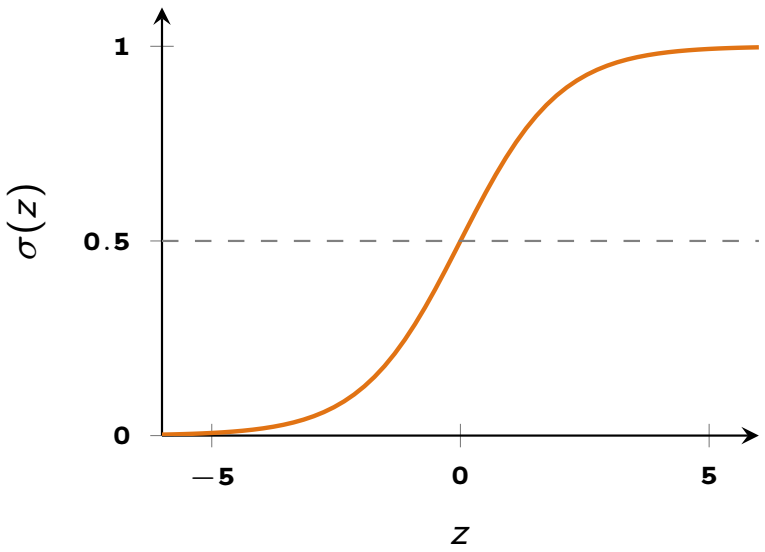
# Regressão logística

Apesar do nome, é um **classificador**:

$$P(y = 1 \mid \mathrm{x}) = \sigma(z), \quad z = \mathrm{w}^\top \mathrm{x} + b.$$

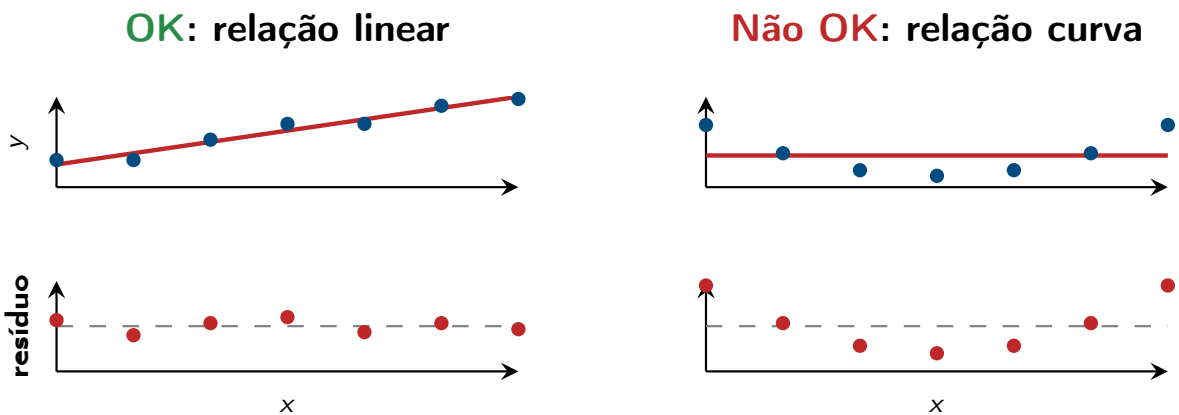
$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Prevemos a classe 1 quando $P \geq 0{,}5$, isto é, quando $\mathrm{w}^\top \mathrm{x} + b \geq 0$. A perda é a entropia cruzada.



# Suposições da regressão linear

| Suposição | Como verificar | Possível remédio |
|---|---|---|
| Linearidade (nos parâmetros) | resíduos contra valores previstos | transformar variáveis ou incluir termos polinomiais |




Esquerda: os resíduos são aleatórios em torno de zero. Direita: os resíduos formam uma curva em U, indicando não linearidade.

# Exercício

Ajuste uma reta aos pontos $(1,2),(2,3),(3,5),(4,6)$ e preveja $y$ para $x = 5$.

## Resposta

$\bar{x} = 2{,}5$, $\bar{y} = 4$, $\hat{\beta}_1 = 7/5 = 1{,}4$, $\hat{\beta}_0 = 0{,}5$. Previsão: $\hat{y}(5) = 7{,}5$. Os resíduos são $0{,}1$, $-0{,}3$, $0{,}3$ e $-0{,}1$, logo $\mathrm{SSE} = 0{,}2$ e $R^2 = 1 - 0{,}2/10 = 0{,}98$.



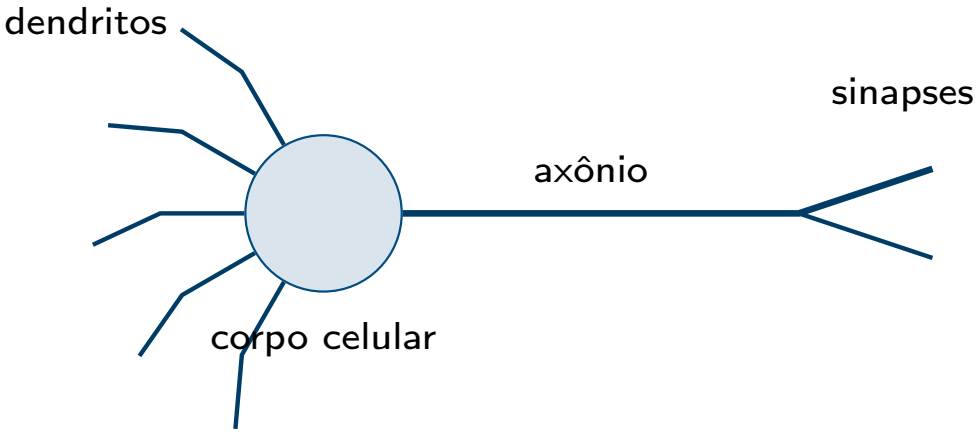
# Exercício

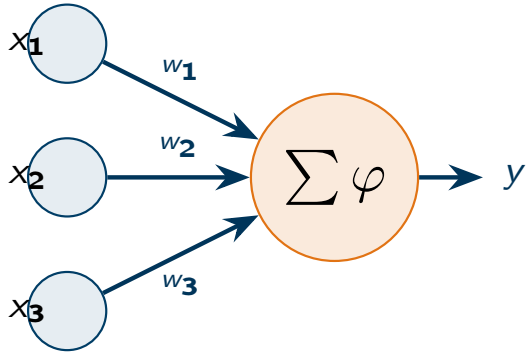
Ajuste uma reta aos pontos $(1,2),(2,3),(3,5),(4,6)$ e preveja $y$ para $x = 5$.

## Resposta

$\bar{x} = 2{,}5$, $\bar{y} = 4$, $\hat{\beta}_1 = 7/5 = 1{,}4$, $\hat{\beta}_0 = 0{,}5$. Previsão: $\hat{y}(5) = 7{,}5$. Os resíduos são $0{,}1$, $-0{,}3$, $0{,}3$ e $-0{,}1$, logo SSE $= 0{,}2$ e $R^2 = 1 - 0{,}2/10 = 0{,}98$.

# Do neurônio biológico ao artificial




O neurônio artificial calcula $y = \varphi(\mathrm{w}^\top \mathrm{x} + b)$. A analogia biológica é inspiração e não um modelo fiel do cérebro.

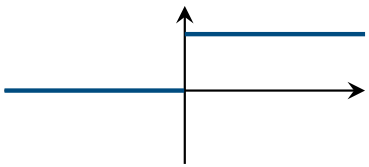
# Neurônio biológico e artificial

| | **Biológico** | **Artificial** |
|---|---|---|
| Entrada | sinais nas sinapses | vetor x |
| Força da conexão | eficácia sináptica | peso $w_i$ |
| Disparo | potencial passa de um limiar | função de ativação $\varphi$ |
| Escala | cerca de $10^{11}$ neurônios, até $10^4$ conexões cada | de milhares a bilhões de parâmetros |

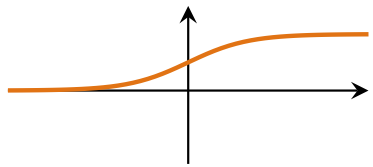


# Funções de ativação

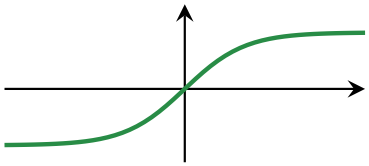
**Degrau**



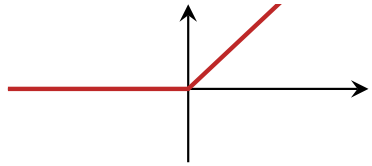
**Sigmoide**



**Tanh**



**ReLU**



# Por que a não linearidade importa

Sem ativação não linear, camadas empilhadas colapsam em uma só:

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2).$$

| **Função** | **Uso típico** | **Observação** |
|---|---|---|
| Sigmoide | saída binária | satura e atenua gradientes |
| Tanh | camadas ocultas antigas | centrada em zero |
| ReLU | camadas ocultas modernas | simples e rápida |
| Softmax | saída multiclasse | produz probabilidades |

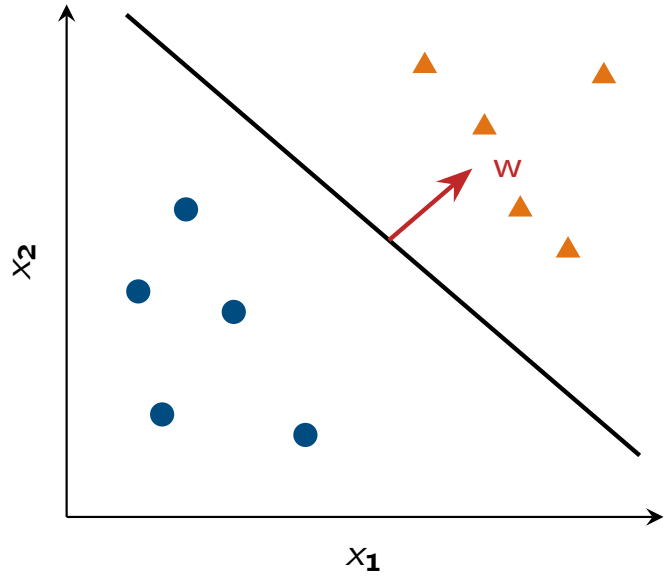


# O Perceptron

Proposto por Rosenblatt (1958). Sobre o modelo de McCulloch e Pitts (1943), ele aprende os pesos:

$$\hat{y} = \begin{cases} 1, & \mathrm{w}^\top \mathrm{x} + b \geq 0 \\ 0, & \text{caso contrário.} \end{cases}$$

Geometricamente, a equação $\mathrm{w}^\top \mathrm{x} + b = 0$ é uma reta que separa as classes.



# OR e AND com um neurônio

Com $u = w_1 x_1 + w_2 x_2 + b$ e saída 1 se $u \geq 0$:

| $x_1$ | $x_2$ | OR: $w = (1,1)$, $b = -0{,}5$ | | AND: $w = (1,1)$, $b = -1{,}5$ | |
|---|---|---|---|---|---|
| | | $u$ | $y$ | $u$ | $y$ |
| 0 | 0 | $-0{,}5$ | 0 | $-1{,}5$ | 0 |
| 0 | 1 | 0,5 | 1 | $-0{,}5$ | 0 |
| 1 | 0 | 0,5 | 1 | $-0{,}5$ | 0 |
| 1 | 1 | 1,5 | 1 | 0,5 | 1 |

O viés $b$ desloca a reta e faz o papel do antigo limiar $\theta = -b$.



# Regra de aprendizagem do Perceptron

Para cada exemplo $(\mathrm{x}, y)$ com previsão $\hat{y}$:

$$\mathrm{w} \leftarrow \mathrm{w} + \eta\,(y - \hat{y})\,\mathrm{x}, \qquad b \leftarrow b + \eta\,(y - \hat{y}).$$

- Se acertou, nada muda.
- Se errou, a reta se move em direção ao exemplo mal classificado.
- Se os dados são linearmente separáveis, o algoritmo converge em um número finito de passos.



# Exemplo resolvido: aprendendo AND

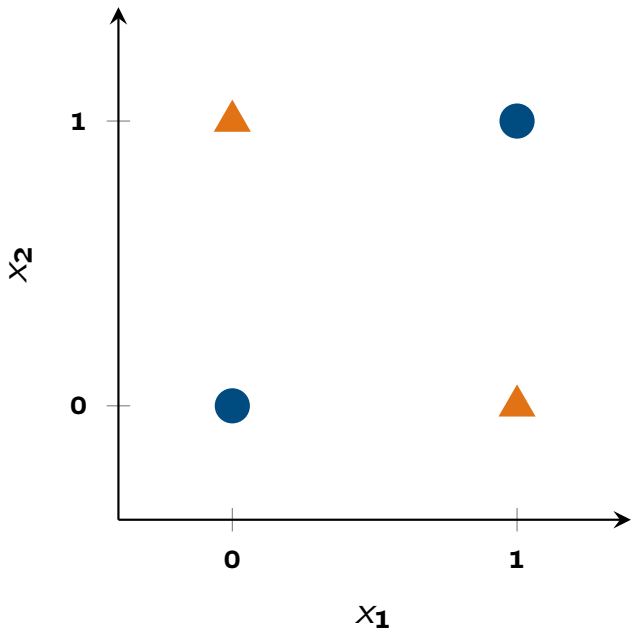
Início: $w_1 = w_2 = 0{,}2$, $b = -0{,}5$ e $\eta = 0{,}1$.

| $x_1$ | $x_2$ | alvo | $u$ | $\hat{y}$ | erro | novos $w$ e $b$ |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | $-0{,}5$ | 0 | 0 | sem mudança |
| 0 | 1 | 0 | $-0{,}3$ | 0 | 0 | sem mudança |
| 1 | 0 | 0 | $-0{,}3$ | 0 | 0 | sem mudança |
| 1 | 1 | 1 | $-0{,}1$ | 0 | $+1$ | $w_1 = w_2 = 0{,}3$, $b = -0{,}4$ |
| 1 | 1 | 1 | 0,2 | 1 | 0 | convergiu |

Na segunda passada, os quatro exemplos já são classificados corretamente.



# O problema do XOR



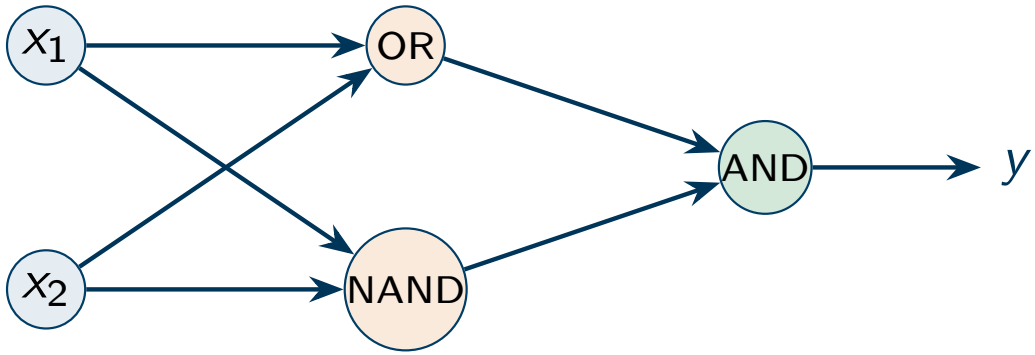

Nenhuma reta separa as classes. Por contradição, seriam necessárias:

- $b < 0$ e $w_1 + w_2 + b < 0$,
- $w_1 + b \geq 0$ e $w_2 + b \geq 0$.

Somando as duas últimas:
$w_1 + w_2 + 2b \geq 0$, logo
$w_1 + w_2 + b \geq -b > 0$. Isso contradiz a segunda desigualdade da primeira linha.

Minsky e Papert (1969) apontaram esse limite.

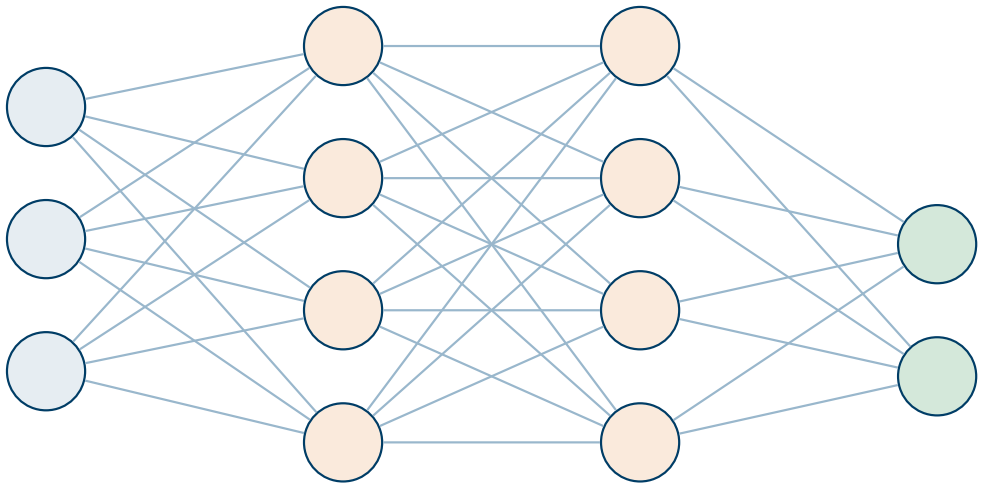
# Duas camadas resolvem o XOR




| Neurônio | Pesos | Viés |
|---|---|---|
| OR | $(1, 1)$ | $-0{,}5$ |
| NAND | $(-1, -1)$ | 1,5 |
| AND (saída) | $(1, 1)$ | $-1{,}5$ |

Entrada $(1, 0)$: OR $= 1$, NAND $= 1$, saída 1. Entrada $(1, 1)$: OR $= 1$, NAND $= 0$, saída 0.

# Perceptron multicamadas (MLP)



entrada camadas ocultas saída

Cada camada densa calcula $L(\mathrm{x}) = \varphi(W\mathrm{x} + \mathrm{b})$, com $W \in \mathbb{R}^{m \times n}$ para $n$ entradas e $m$ neurônios.



# Composição de camadas

$$f(\mathrm{x}) = \varphi_2\big(W_2\,\varphi_1(W_1\mathrm{x} + \mathrm{b}_1) + \mathrm{b}_2\big).$$

- As primeiras camadas extraem atributos simples (bordas, texturas).
- As camadas seguintes os combinam em conceitos (partes, objetos).
- Chamamos de **profunda** a rede com duas ou mais camadas ocultas.
- **Teorema da aproximação universal**: uma rede com uma camada oculta suficientemente larga aproxima qualquer função contínua em um conjunto compacto. O teorema garante a existência, mas não ensina a encontrar os pesos.



# Função de perda

Treinar significa minimizar uma perda média sobre os dados:

$$\text{Regressão (MSE):} \quad \mathcal{L} = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

$$\text{Classificação binária:} \quad \mathcal{L} = -\frac{1}{m}\sum_{i=1}^{m}\left[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\right]$$

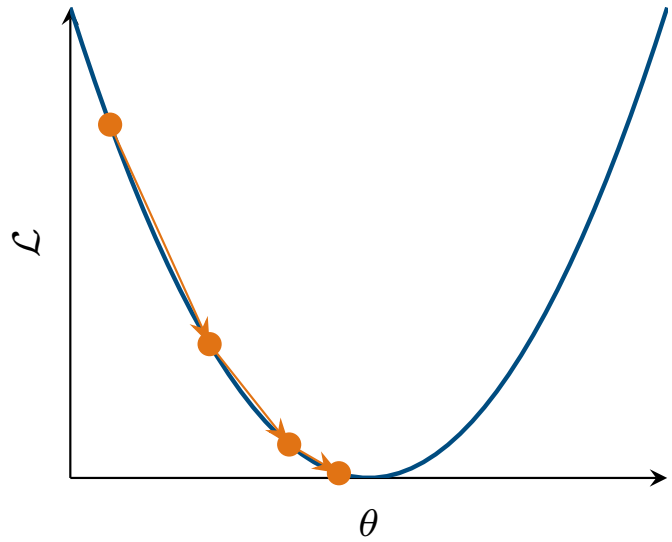
$$\text{Multiclasse:} \quad \mathcal{L} = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_{ik} \log \hat{y}_{ik}$$



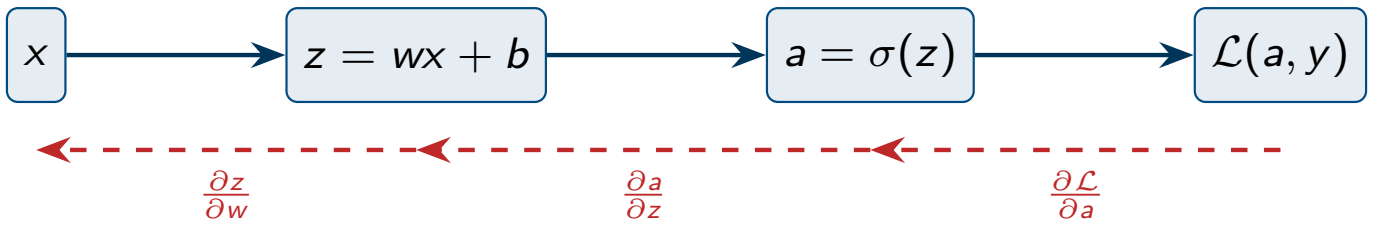
# Gradiente descendente

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$$

- $\eta$ é a taxa de aprendizado.
- *Mini-batch*: o gradiente usa um subconjunto dos dados por passo.
- Uma *época* percorre todos os dados uma vez.
- Otimizadores como Adam adaptam o passo a cada parâmetro.



# Retropropagação




$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

A fase *forward* calcula as saídas. A fase *backward* propaga o erro com a regra da cadeia, reaproveitando cálculos de cada camada.

# Um passo de treinamento, com números

Neurônio sigmoide com $x = 1$, $w = 0{,}5$, $b = 0$, alvo $y = 1$ e perda $\mathcal{L} = \frac{1}{2}(a - y)^2$.

$$a = \sigma(0{,}5) \approx 0{,}6225, \qquad \mathcal{L} \approx 0{,}0713,$$

$$\frac{\partial \mathcal{L}}{\partial w} = (a - y)\, a(1 - a)\, x \approx -0{,}0887,$$

$$w_{\text{novo}} = 0{,}5 - \eta \cdot (-0{,}0887) \approx 0{,}589 \quad (\eta = 1).$$

O peso aumenta, a saída se aproxima de 1 e a perda diminui.

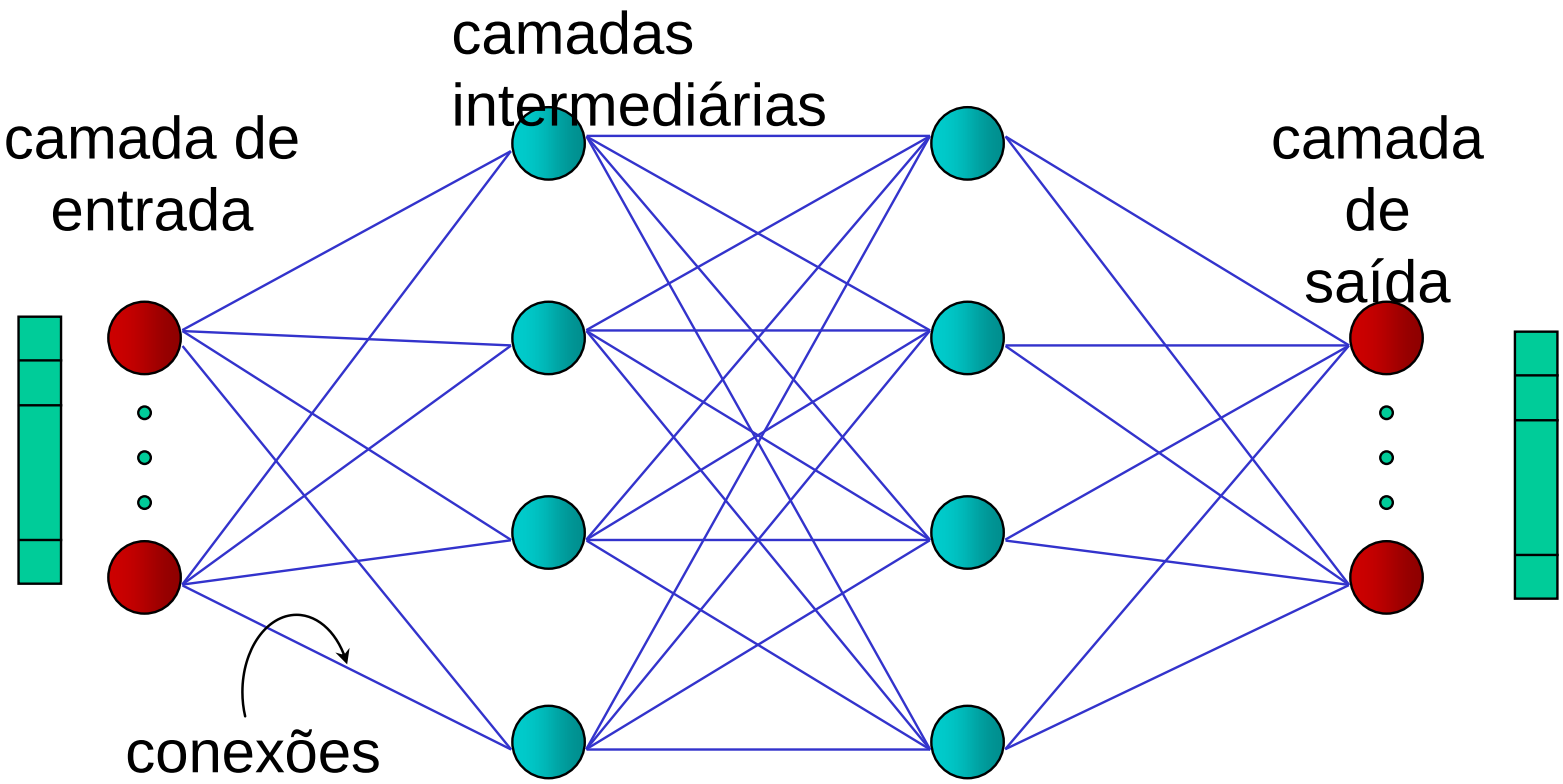


# Animação do aprendizado

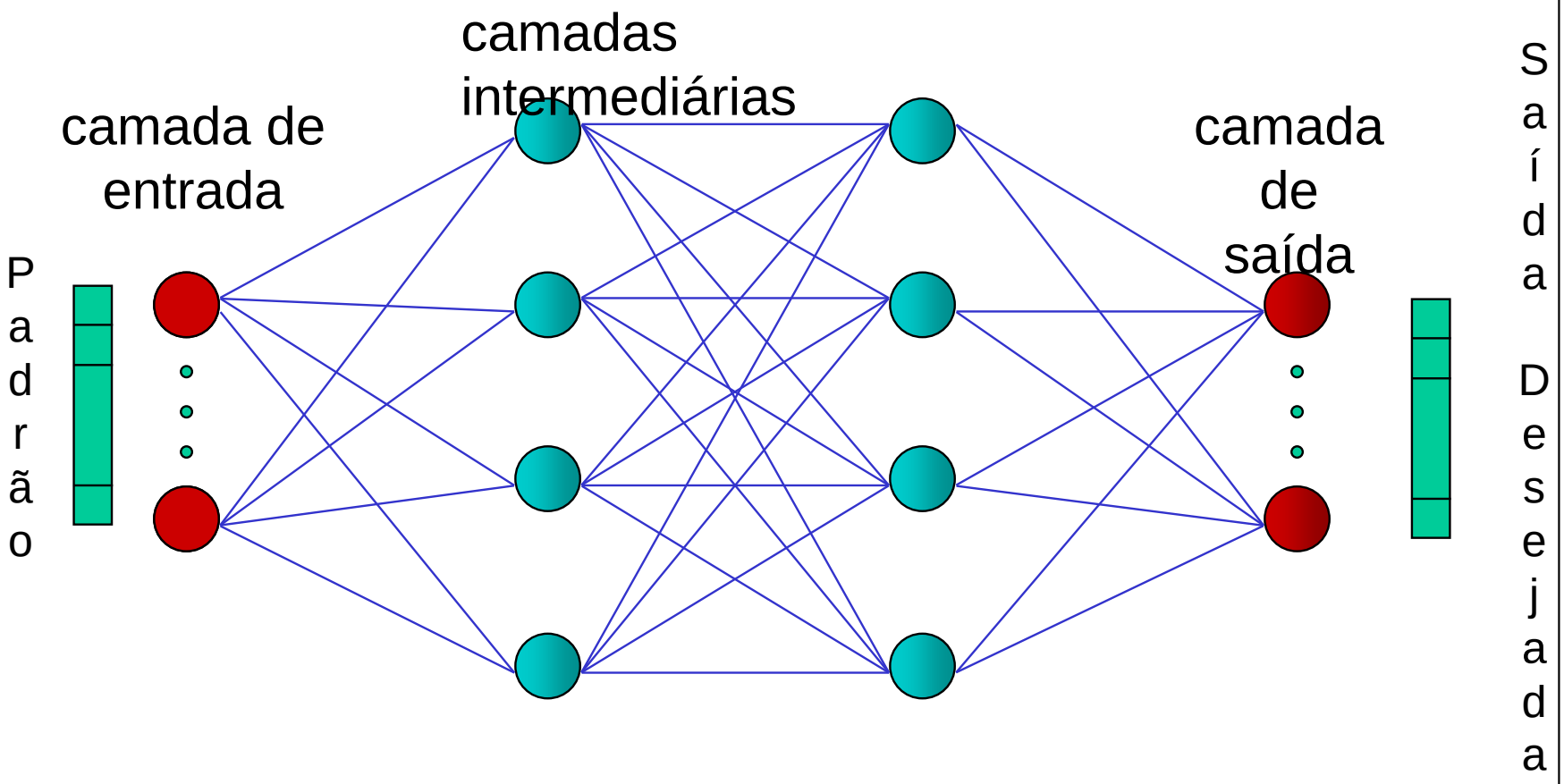
A seguir, uma sequência de quadros mostra a rede ajustando seus pesos.



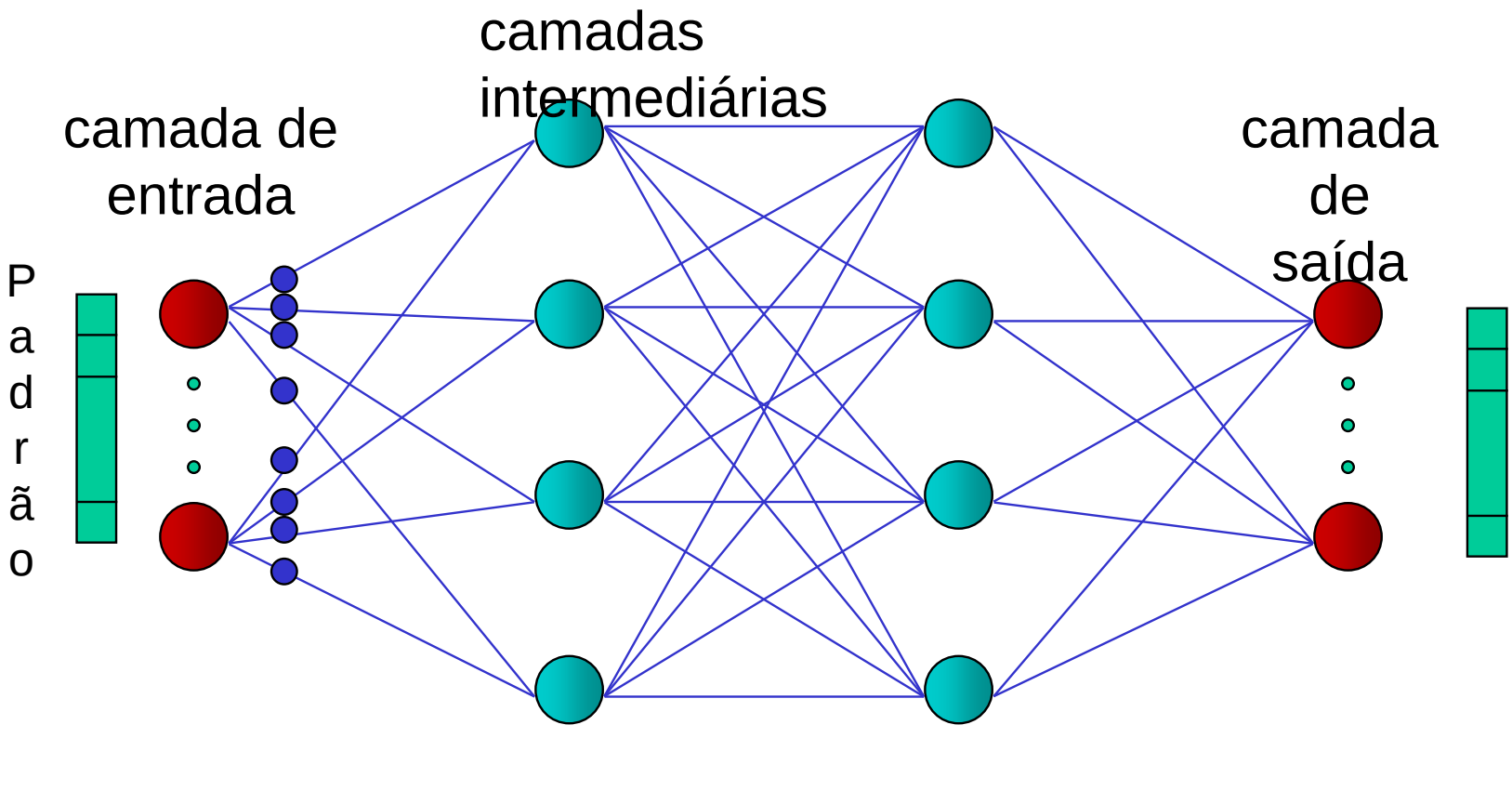
# Rede MLP


camadas
intermediárias
camada de
entrada
camada
de
saída
conexões

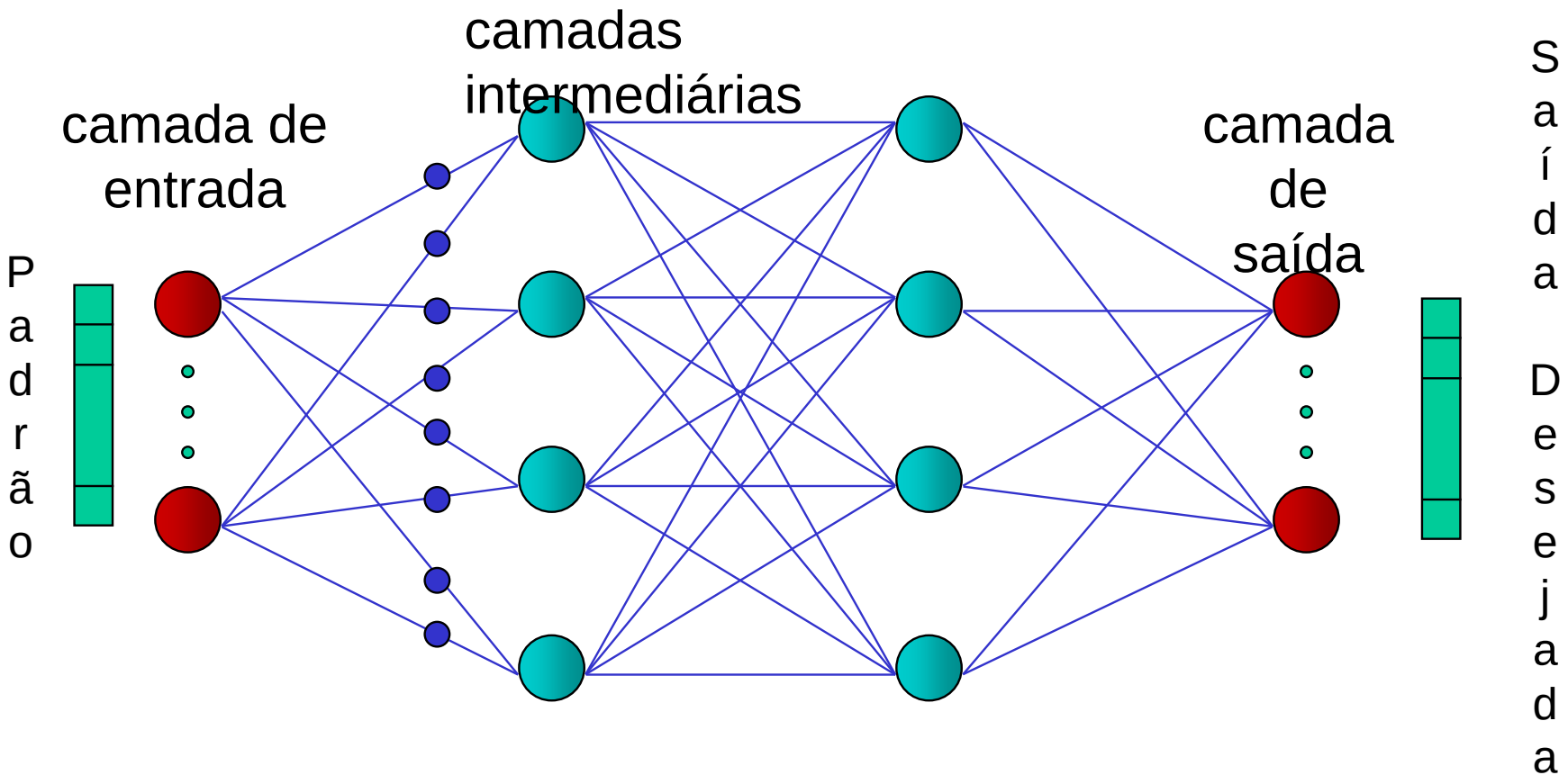
# Aprendizado



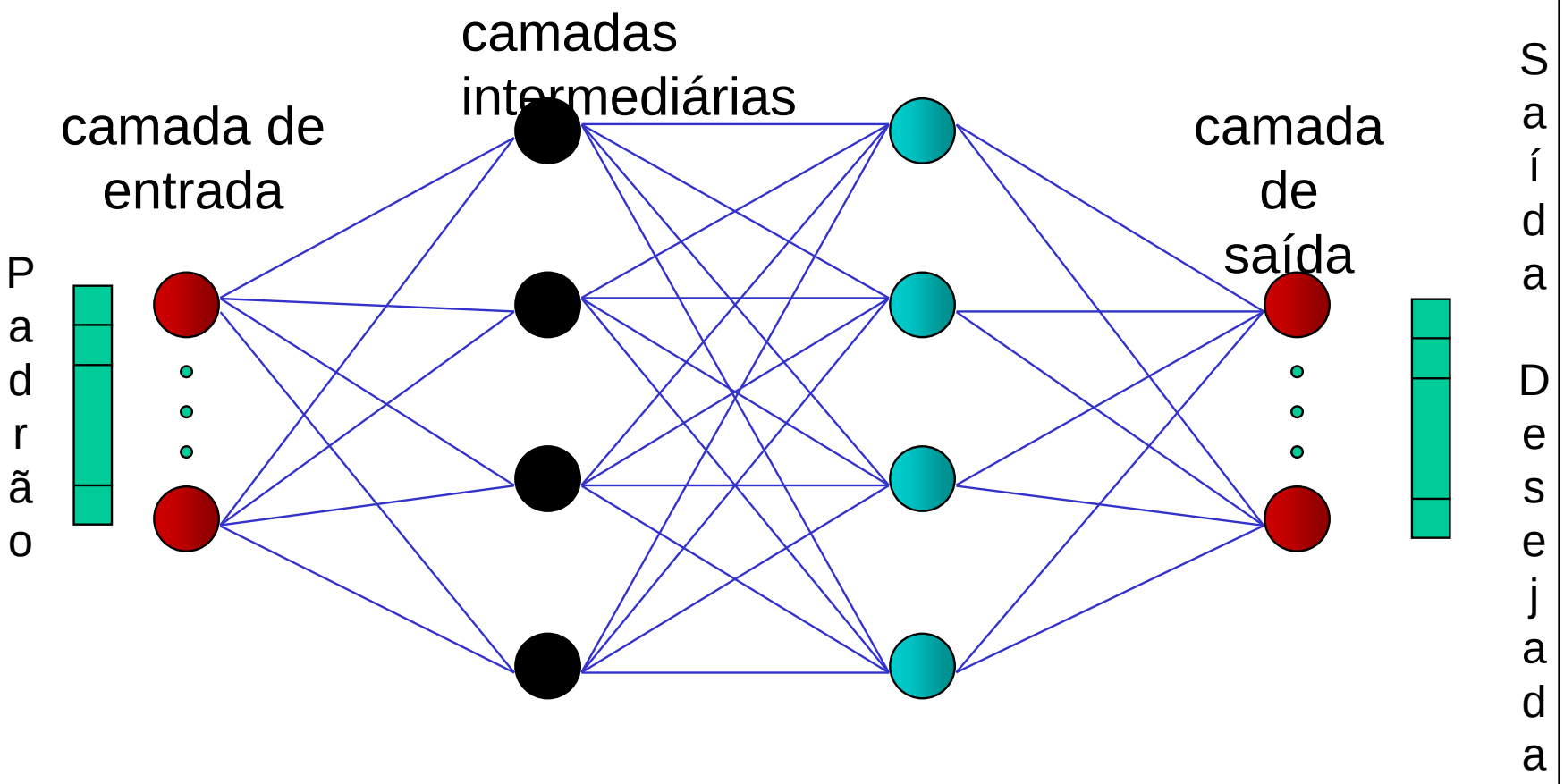
# RNA - Aprendizado



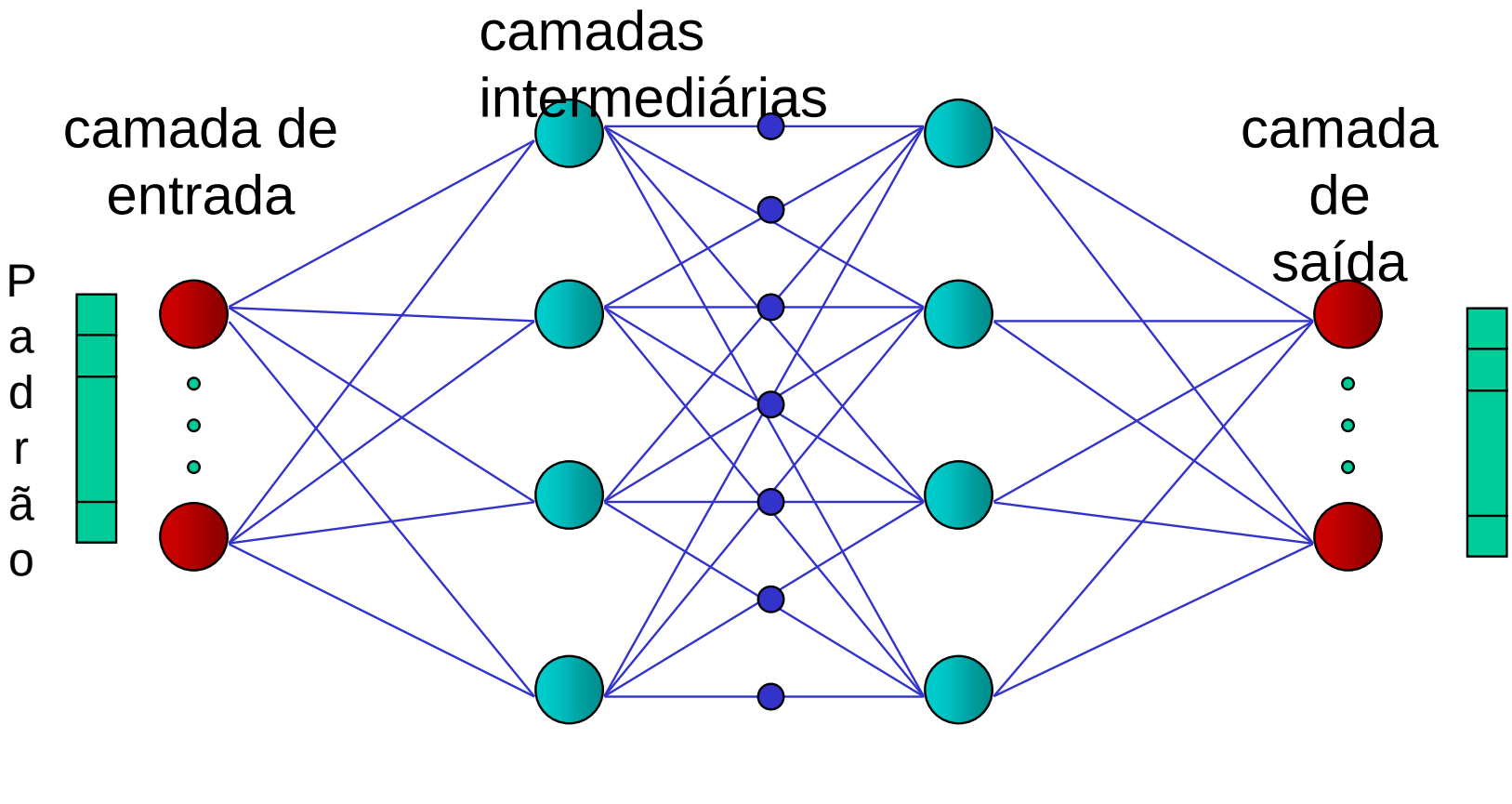
# RNA - Aprendizado



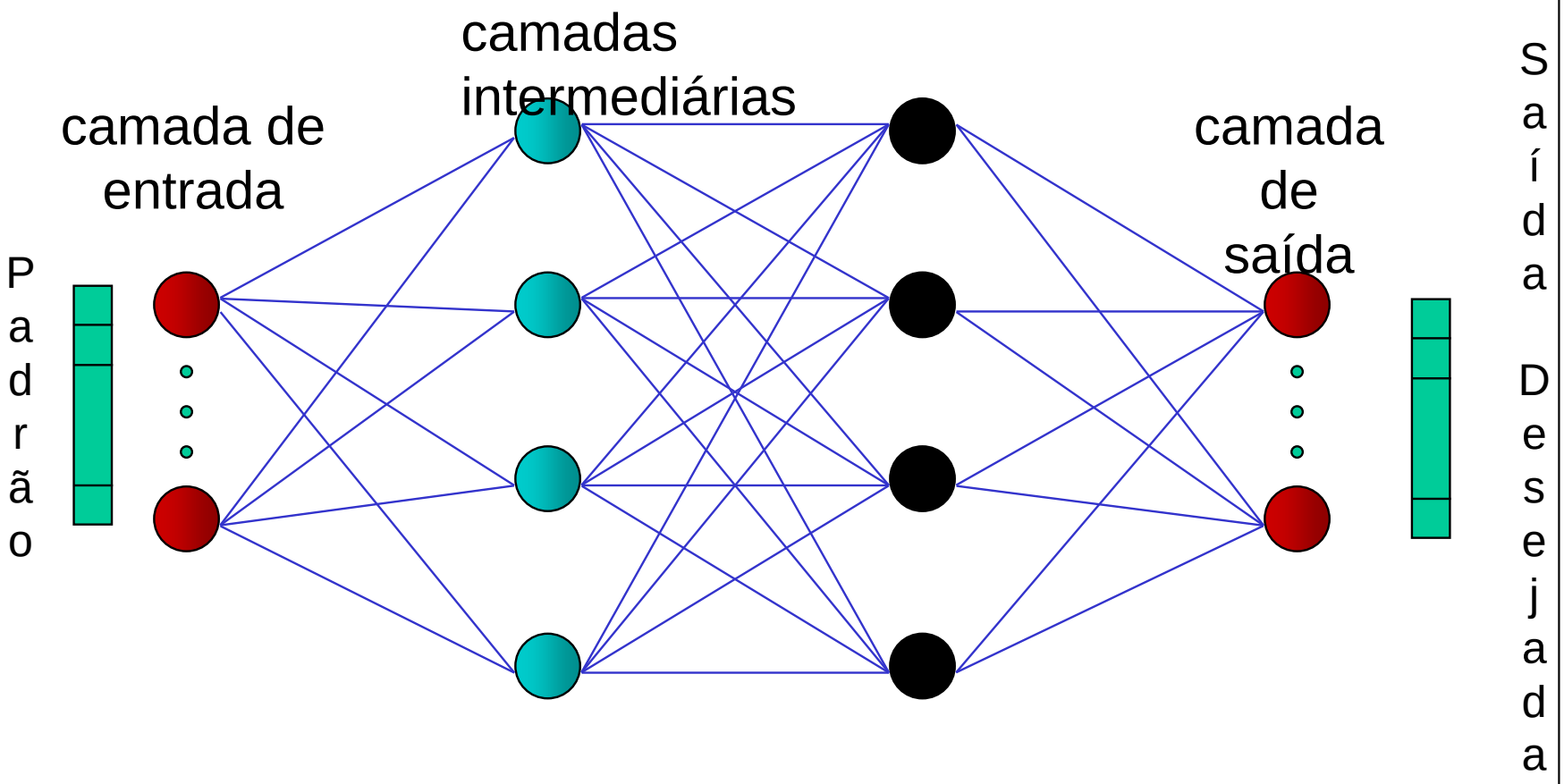
# RNA - Aprendizado



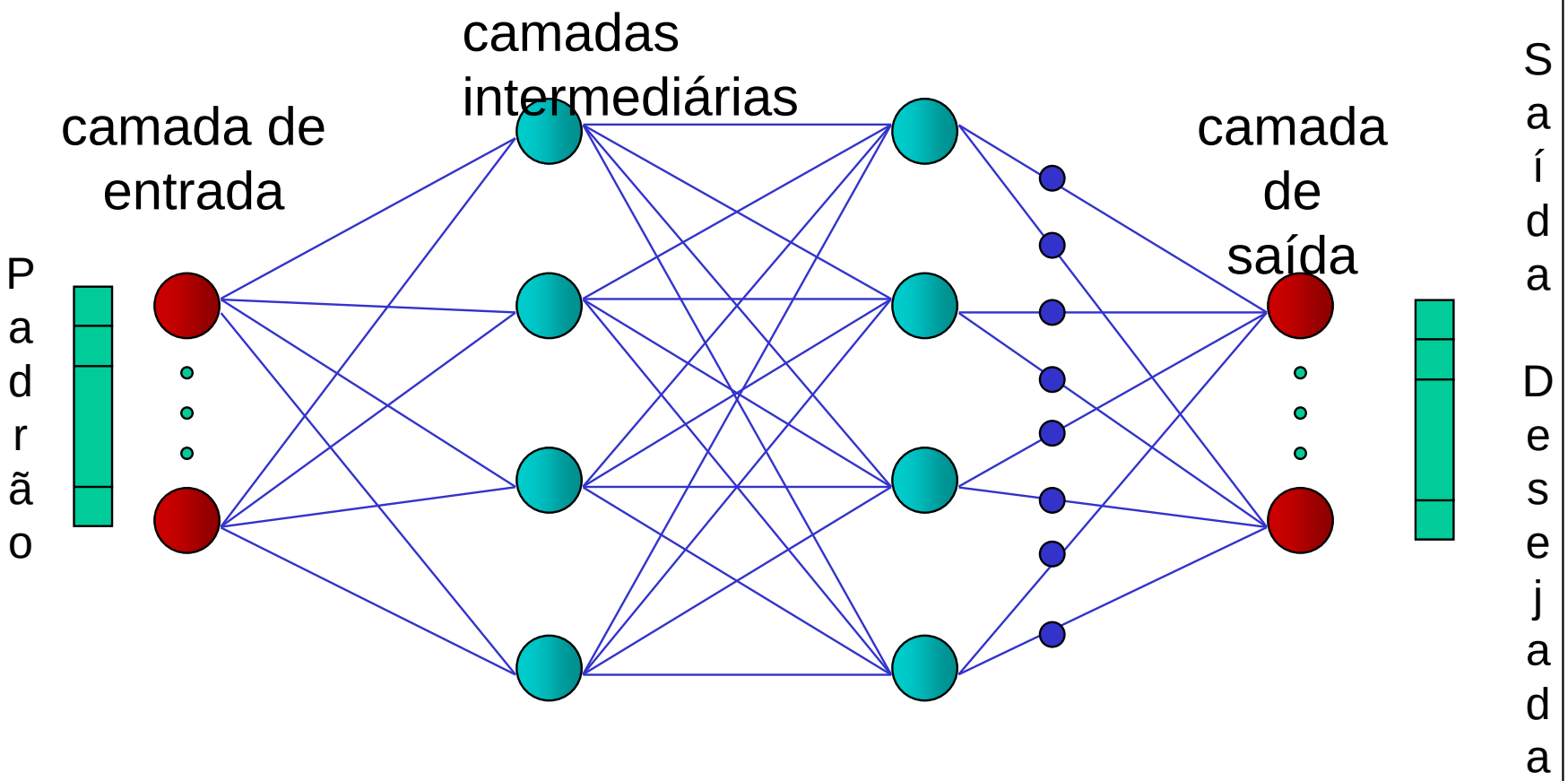
# RNA - Aprendizado



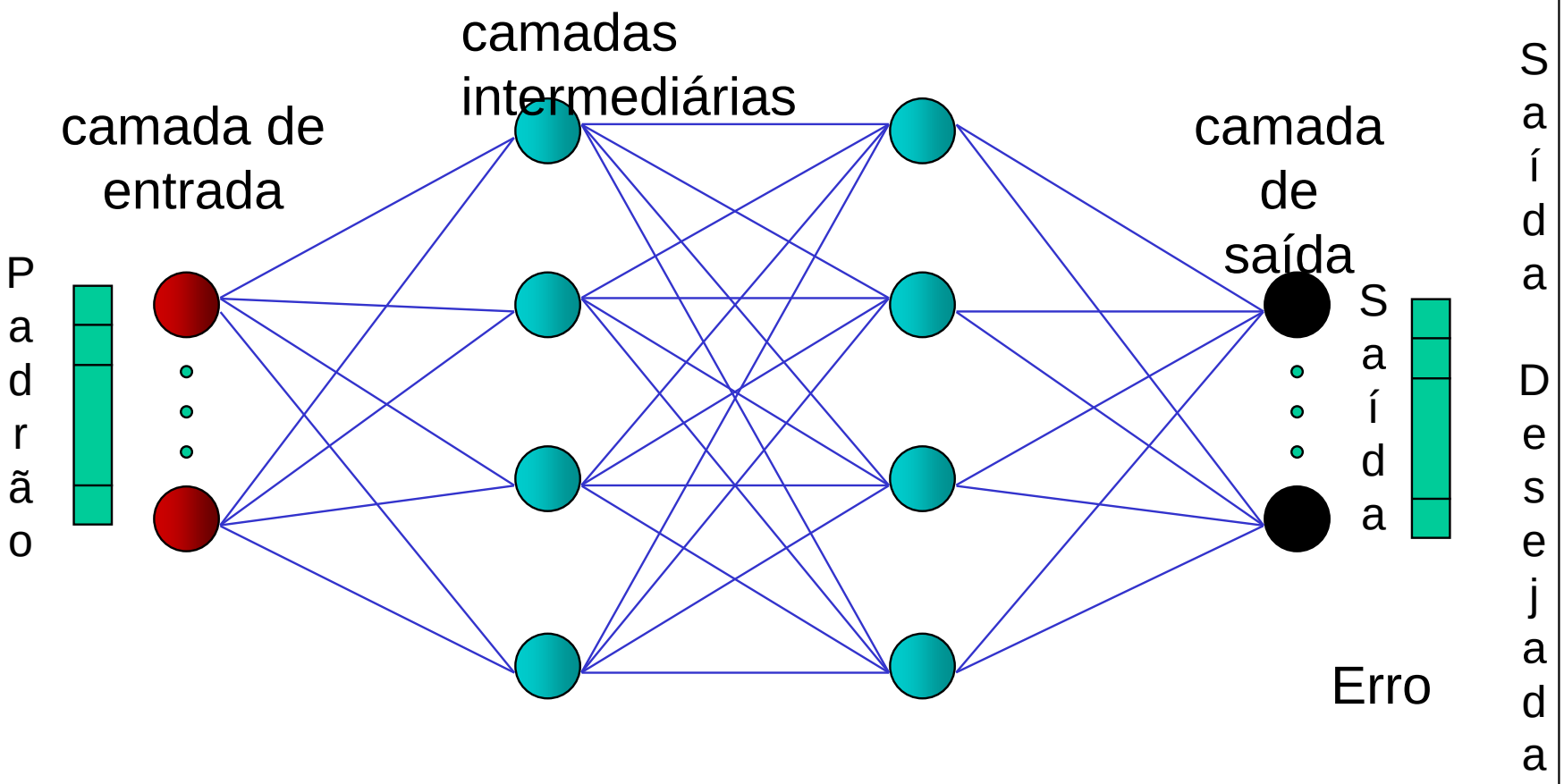
# RNA - Aprendizado



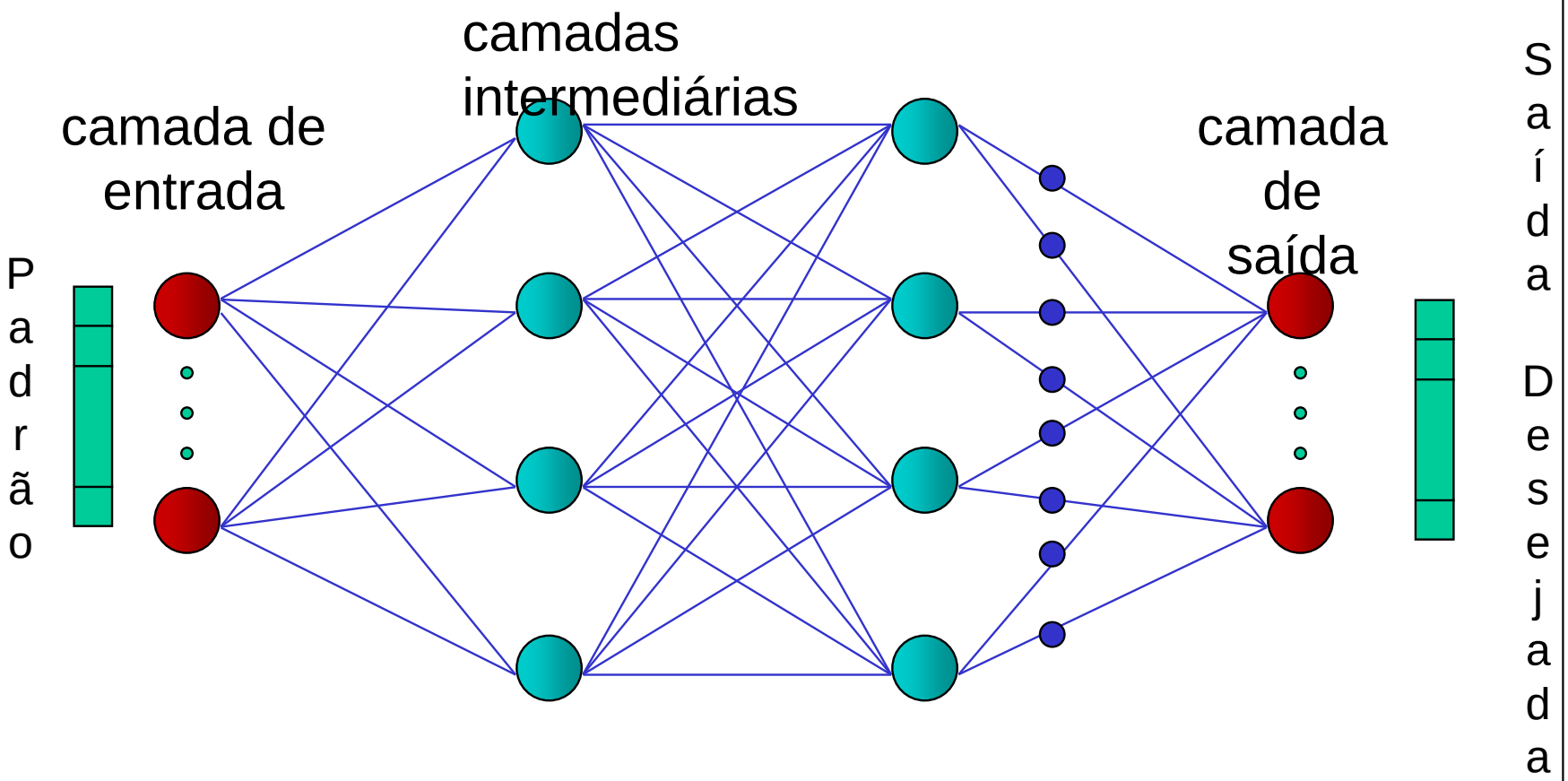
# RNA - Aprendizado



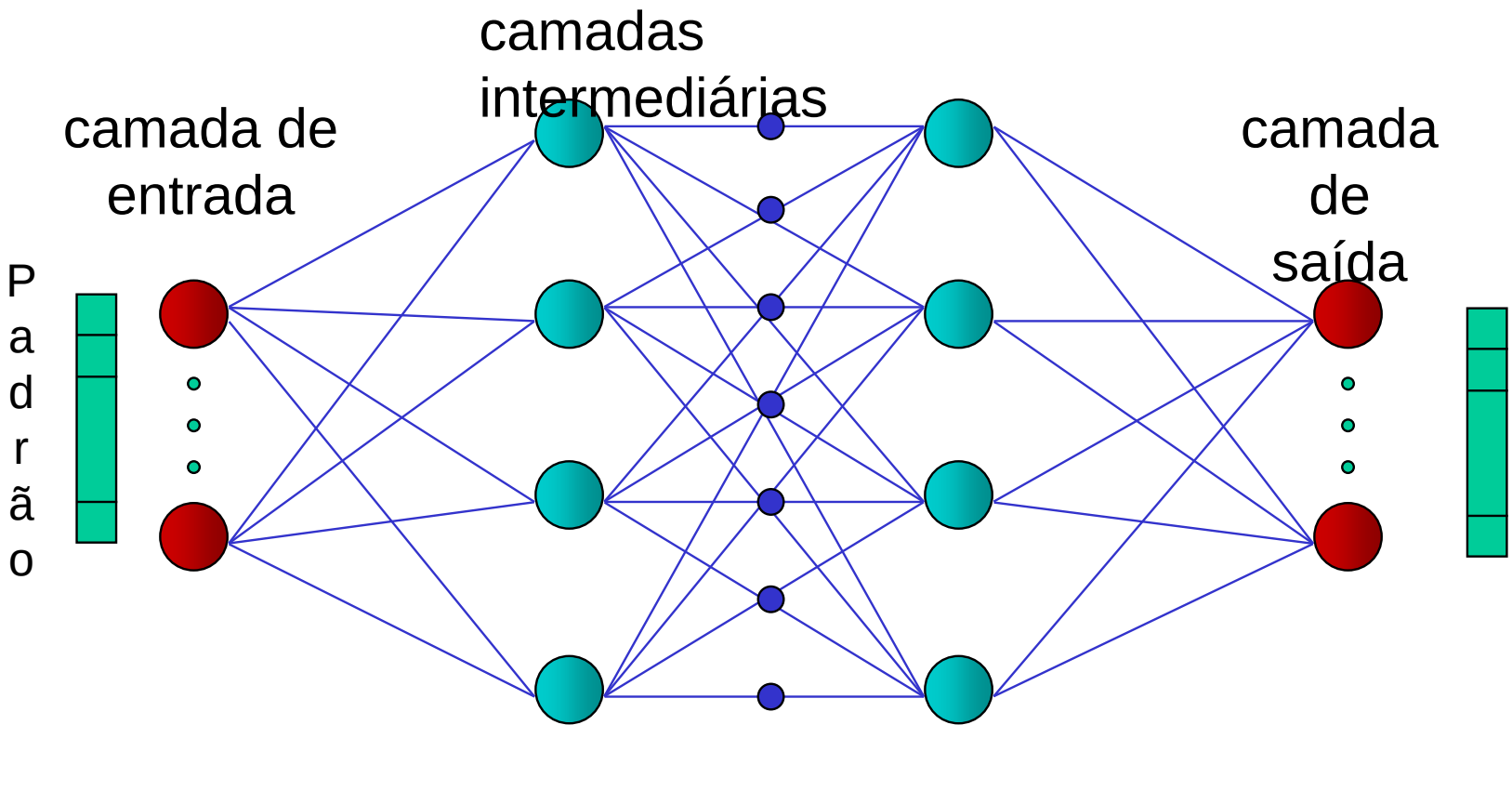
# RNA - Aprendizado



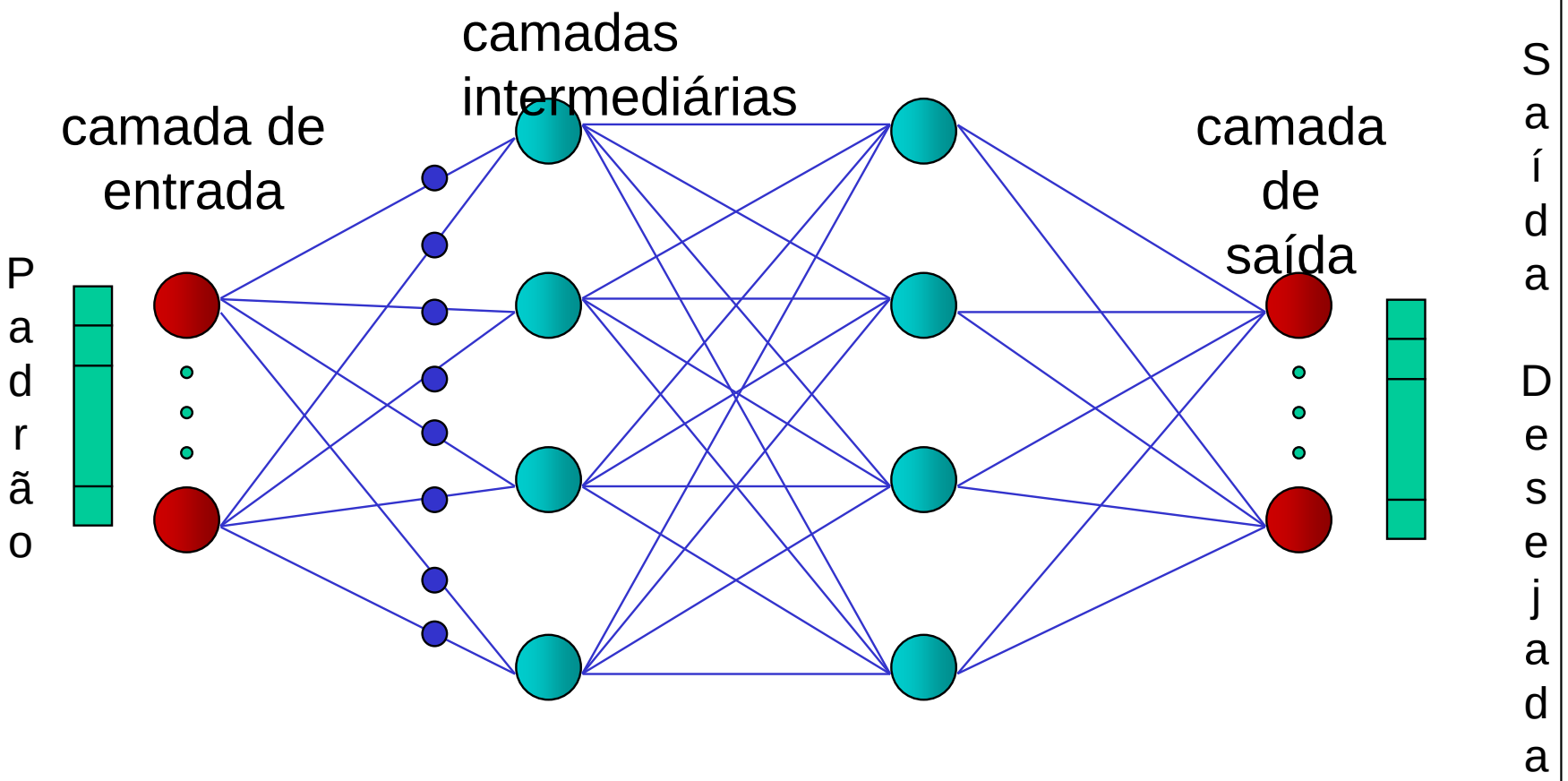
# RNA - Aprendizado



# RNA - Aprendizado

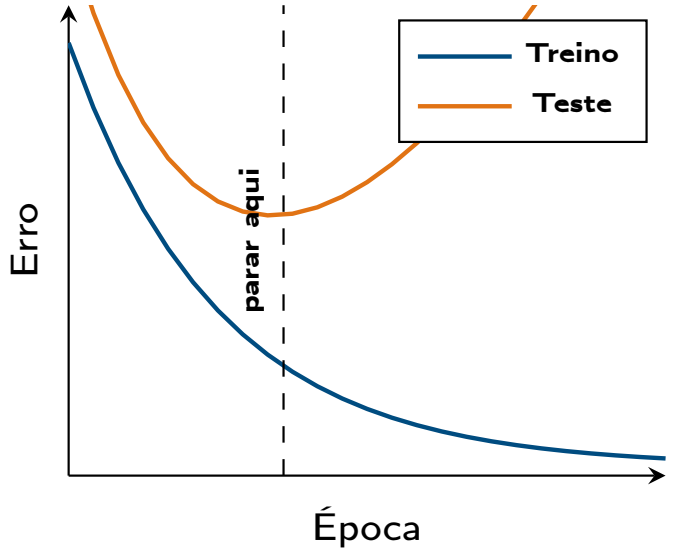


# RNA - Aprendizado



# Overfitting em redes neurais

- **Parada antecipada**: interrompa quando o erro de teste deixa de cair.
- **Regularização L2** nos pesos.
- ***Dropout***: desliga neurônios aleatoriamente no treino.
- **Aumento de dados**.



# Redes na prática

```python
import numpy as np
from sklearn.neural_network import MLPClassifier

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0, 1, 1, 0])              # XOR

mlp = MLPClassifier(hidden_layer_sizes=(4,), activation="tanh",
                    solver="lbfgs", max_iter=2000, random_state=0)
mlp.fit(X, y)
print(mlp.predict(X))                   # [0 1 1 0]
```

Para redes maiores, use PyTorch ou Keras, que treinam em GPU.



# Arquiteturas atuais

| **Arquitetura** | **Dados típicos** | **Exemplo de uso** |
|---|---|---|
| Convolucional (CNN) | imagens (ver Aula 3) | diagnóstico por imagem |
| Recorrente e LSTM | sequências | séries temporais |
| *Transformer* | texto, imagem, áudio | modelos de linguagem |
| Difusão | imagens e vídeo | geração de imagens |

**Limitações**: grande consumo de dados e energia, pouca interpretabilidade e respostas plausíveis porém incorretas.



# Exercício

Use os pesos da rede do XOR e calcule a saída para $x = (0, 0)$.

## Resposta

OR: $u = -0{,}5$, logo $h_1 = 0$. NAND: $u = 1{,}5$, logo $h_2 = 1$. Saída: $u = 0 + 1 - 1{,}5 = -0{,}5$, portanto $y = 0$. Correto.



# Exercício

Use os pesos da rede do XOR e calcule a saída para $x = (0, 0)$.

## Resposta

OR: $u = -0{,}5$, logo $h_1 = 0$. NAND: $u = 1{,}5$, logo $h_2 = 1$. Saída: $u = 0 + 1 - 1{,}5 = -0{,}5$, portanto $y = 0$. Correto.



# Aprendizado sem rótulos

Dado apenas $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$, buscamos **estrutura** nos dados.

- Agrupamento (*clustering*).
- Redução de dimensionalidade.
- Detecção de anomalias.



**Figura:** Amostras sem rótulo agrupadas por semelhança.

**Para refletir**

Como saber se um agrupamento é bom, se não há resposta correta?

# K-Means

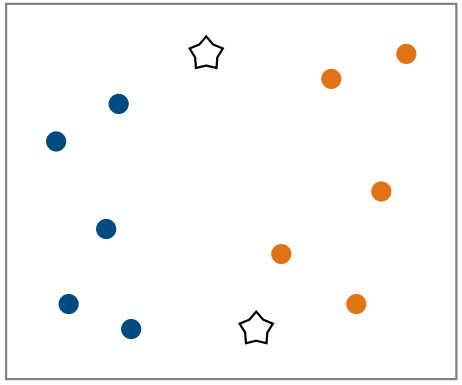
Particiona os dados em $k$ grupos que minimizam a soma dos quadrados intragrupo:

$$\mathcal{J} = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2.$$

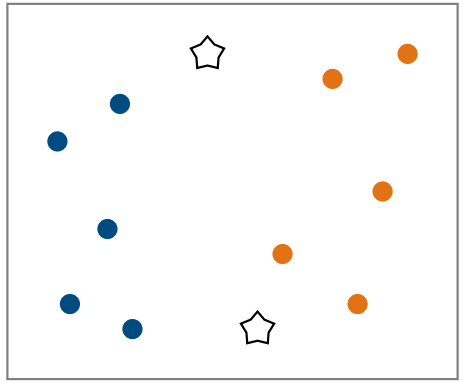
1. Escolha $k$ centróides iniciais.
2. Atribua cada ponto ao centróide mais próximo.
3. Recalcule cada centróide como a média do seu grupo.
4. Repita até as atribuições não mudarem.



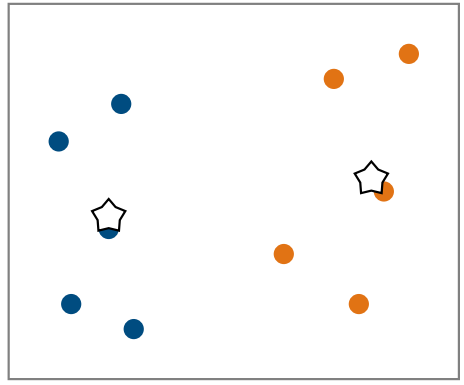
# K-Means em três passos



1. Início



2. Atribuir



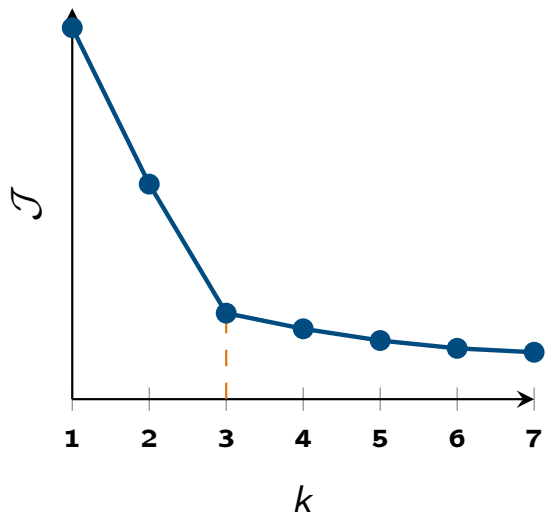
3. Atualizar

# Limitações e escolha de $k$

- O resultado depende da inicialização, e por isso se usa *k-means++*.
- Assume grupos aproximadamente esféricos.
- Exige atributos na mesma escala.
- **Escolha de** $k$: método do cotovelo e coeficiente de silhueta

$$s = \frac{b - a}{\max(a, b)}$$

$a$: distância média ao próprio grupo. $b$: distância média ao grupo mais próximo.

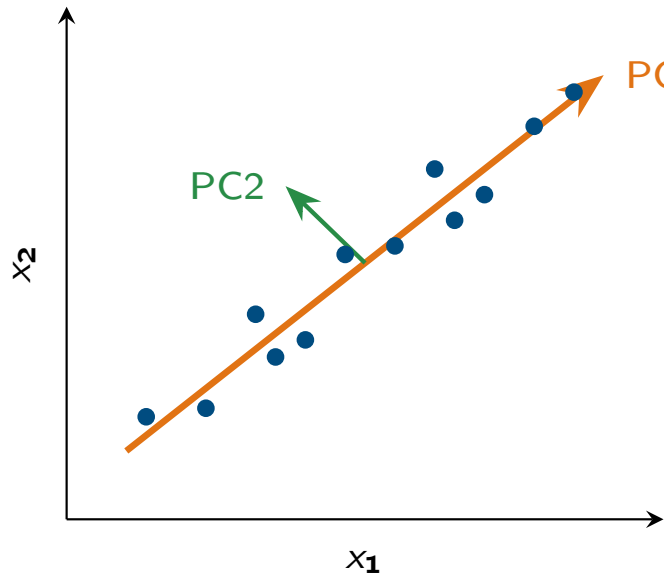



Curva ilustrativa. O cotovelo sugere $k = 3$.

# Análise de Componentes Principais (PCA)

Projeta os dados nas direções de **maior variância**.

1. Centralize os dados.
2. Calcule a matriz de covariância $\Sigma$.
3. Obtenha autovalores e autovetores, $\Sigma v = \lambda v$.
4. Projete nos $k$ autovetores de maiores autovalores.



# PCA: exemplo resolvido

Com $\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, os autovalores são $\lambda_1 = 3$ e $\lambda_2 = 1$, com autovetores

$$v_1 = \frac{1}{\sqrt{2}}(1, 1), \qquad v_2 = \frac{1}{\sqrt{2}}(1, -1).$$

A primeira componente explica $\dfrac{3}{3+1} = 75\%$ da variância total.

Usos: visualizar dados em duas dimensões, remover redundância e acelerar outros modelos. Para visualização não linear, existem t-SNE e UMAP.

**Para refletir**

Que informação se perde ao manter apenas PC1?



# Exercício

Aplique o K-Means ($k = 2$) aos pontos $1, 3, 4, 10$ com centróides iniciais $\mu_1 = 1$ e $\mu_2 = 10$.

## Resposta

Atribuição: $C_1 = \{1, 3, 4\}$ e $C_2 = \{10\}$. Novos centróides: $\mu_1 = 8/3 \approx 2{,}67$ e $\mu_2 = 10$. A atribuição se repete, então o algoritmo convergiu.



# Exercício

Aplique o K-Means ($k = 2$) aos pontos $1, 3, 4, 10$ com centróides iniciais $\mu_1 = 1$ e $\mu_2 = 10$.

## Resposta

Atribuição: $C_1 = \{1, 3, 4\}$ e $C_2 = \{10\}$. Novos centróides: $\mu_1 = 8/3 \approx 2{,}67$ e $\mu_2 = 10$. A atribuição se repete, então o algoritmo convergiu.



# Responsabilidade ao usar ML

- **Viés nos dados** se transforma em viés nas decisões. Em 2018, a Reuters relatou que a Amazon abandonou uma ferramenta de triagem de currículos que penalizava candidatas.
- **Privacidade**: dados pessoais exigem base legal (LGPD) e proteção.
- **Explicabilidade**: decisões de alto impacto precisam de justificativa.
- **Mudança de contexto**: modelos degradam quando o mundo muda.

**? Para refletir**

Quem responde quando um modelo erra em uma decisão sobre pessoas?



# Resumo da aula

| **Tema** | **Ideia central** | **Pergunta-chave** |
|---|---|---|
| Supervisionado | aprender $x \mapsto y$ com exemplos | o modelo generaliza? |
| Teste | separar treino e teste | há vazamento de dados? |
| Classificação | matriz de confusão e métricas | qual erro custa mais? |
| Regressão | mínimos quadrados e resíduos | as suposições valem? |
| Redes neurais | camadas, perda e gradiente | o que aprendeu, e com que custo? |
| Não supervisionado | agrupar e reduzir dimensões | a estrutura faz sentido? |



# Referências I



Russell, S. e Norvig, P. (2021). *Artificial Intelligence: A Modern Approach*. 4ª ed. Pearson.


Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3ª ed. O'Reilly.


Goodfellow, I., Bengio, Y. e Courville, A. (2016). *Deep Learning*. MIT Press.


Chollet, F. (2021). *Deep Learning with Python*. 2ª ed. Manning.


James, G. et al. (2021). *An Introduction to Statistical Learning*. 2ª ed. Springer.


Mitchell, T. (1997). *Machine Learning*. McGraw-Hill.


McCulloch, W. e Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5.


Rosenblatt, F. (1958). The perceptron. *Psychological Review*, 65(6).




# Referências II


Rumelhart, D., Hinton, G. e Williams, R. (1986). Learning representations by back-propagating errors. *Nature*, 323.


Vaswani, A. et al. (2017). Attention is all you need. *NeurIPS*.






# Machine Learning

Professor: Elton Sarmanho[1]
E-mail: eltonss@ufpa.br






[1]Faculdade de Sistemas de Informação - UFPA/CUNTINS

21 de outubro de 2026